Red Hat OpenShift AI Self-Managed 3.4

Govern LLM access with Models-as-a-Service

Govern LLM access with Models-as-a-Service in Red Hat OpenShift AI Self-Managed

Last Updated: 2026-06-11

Red Hat OpenShift AI Self-Managed 3.4 Govern LLM access with Models-as-a-Service

Govern LLM access with Models-as-a-Service in Red Hat OpenShift AI Self-Managed

Legal Notice

Abstract

As a Red Hat OpenShift AI user, you can deploy Models-as-a-Service (MaaS) in Red Hat OpenShift AI Self-Managed to provide governed access to large language models across your organization.

Table of Contents

CHAPTER 1. DEPLOY AND MANAGE MODELS-AS-A-SERVICE

You can deploy Models-as-a-Service (MaaS) to provide subscription-based governance for large language model serving. With MaaS, you can define subscriptions that grant groups access to models with token limits, control access through API key authentication, and track resource consumption for cost allocation.

1.1. MODELS-AS-A-SERVICE OVERVIEW

In Red Hat OpenShift AI, Models-as-a-Service (MaaS) provides subscription-based governance for large language model (LLM) serving across your organization. This platform helps you manage resource consumption and governance challenges when you serve models to a large user base.

NOTE

In OpenShift AI 3.4, MaaS uses a subscription-based model for quota management. This replaces the tier-based model used in OpenShift AI 3.3. Subscriptions provide enhanced flexibility and are managed through custom resources instead of ConfigMaps.

As an administrator, you can use this subscription-based system to expose models through managed API endpoints. With this structure, you can enforce different consumption policies for different user groups and deliver AI models as shared resources with appropriate access levels.

The Models-as-a-Service platform acts as a governance layer between users and model serving infrastructure. You can enforce centralized policies without modifying the underlying model serving components.

Models-as-a-Service provides the following capabilities:

Subscription-based quota management

Define multiple subscriptions that grant specific groups quota for models with configurable token limits. Users can belong to multiple subscriptions, with priority levels determining which subscription is used when multiple options are available.

Self-service API key management

Users can create, list, and revoke their own API keys for model access. Administrators can also provision and manage API keys on behalf of users. API keys can be permanent or configured with custom expiration times, and individual keys can be revoked without affecting other keys for the same user.

Multi-runtime support

Expose models served with llm-d or vLLM runtimes through MaaS governance. You can apply consistent governance across different serving infrastructures. vLLM runtime support is a Technology Preview feature in Red Hat OpenShift AI.

Policy and quota management

Enforce token limit policies to prevent resource exhaustion.

Usage tracking and observability

Monitor subscription-level token consumption, request counts, and rate-limit violations through the MaaS observability dashboard. Track consumption metrics for cost allocation and billing. Export usage data in CSV format for cost attribution and showback reporting to finance teams. The MaaS observability dashboard is a Technology Preview feature in Red Hat OpenShift AI.

External models

Route inference requests to models hosted by external cloud providers such as AWS Bedrock, Azure OpenAI, or Google Vertex AI through the same MaaS gateway used for locally deployed models. External models is a Technology Preview feature in Red Hat OpenShift AI.

External OpenID Connect (OIDC) authentication

Integrate with external OIDC identity providers for user authentication to provide enterprise-wide access without requiring OpenShift user accounts. External OIDC authentication is a Technology Preview feature in Red Hat OpenShift AI.

The following table summarizes when MaaS is the right choice and when standard model serving is sufficient.

Table 1.1. When to use MaaS vs. standard model serving

MaaS	Standard model serving
Centralized governance across multiple teams or projects is required.	You are deploying models for single-team or single-user use cases.
You need token limit enforcement and usage tracking for cost control.	You are prototyping or developing models in a single-user environment where governance overhead is unnecessary.
You prefer declarative configuration management via GitOps.	Simplified deployment is preferred over centralized control.

MaaS administration is divided into initial configuration and ongoing management, with distinct responsibilities for cluster administrators and OpenShift AI administrators.

Table 1.2. MaaS administrator responsibilities

Phase	Cluster administrators	OpenShift AI administrators
Initial configuration	• Enable MaaS in the OpenShift AI operator • Configure the underlying cluster infrastructure to support model serving	• Define the initial governance structure by creating subscriptions and authorization policies • Assign users to groups • Configure model quota and token limits for each subscription • Validate that users can successfully access models through MaaS

Phase	Cluster administrators	OpenShift AI administrators
Ongoing operations	• Scale MaaS components to handle increased load • Apply software updates • Troubleshoot infrastructure performance issues	• Monitor usage metrics to track costs • Adjust subscription configurations and user group assignments • Modify token limits based on demand patterns • Manage API keys for external consumers or assist users with key lifecycle management • Troubleshoot authentication and authorization issues

1.1.1. Models-as-a-Service custom resources

Models-as-a-Service (MaaS) uses Kubernetes custom resources for declarative configuration management. You can integrate MaaS with GitOps workflows and version control. The platform uses the following custom resource types:

Tenant

Configures tenant-specific settings including API key expiration limits, external OIDC authentication, telemetry options, and gateway references.

MaaSModelRef

References inference servers served through OpenShift AI. Models can be served using llm-d distributed inference, vLLM runtimes, or external LLM providers.

ExternalModel

Defines external LLM provider configurations for models hosted outside the cluster, such as OpenAI or Anthropic. You can apply MaaS governance to third-party LLM services.

MaaSSubscription

Defines subscription-based quota by specifying which groups have quota for which models with configurable token rate limits. Subscriptions include priority levels for users belonging to multiple groups and optional metadata for cost allocation.

MaaSAuthPolicy

Authorizes groups to access model endpoints through the API gateway. Subscriptions control token limits, while authorization policies grant API gateway access.

With these custom resources, administrators can manage MaaS configurations using standard Kubernetes tools and GitOps workflows. Changes to custom resources are automatically reconciled by the platform controllers.

You can view and manage these custom resources using the OpenShift console or OpenShift CLI (**oc**).

Using the console:

Navigate to **Administration → CustomResourceDefinitions** and search for the resource name.

Using the CLI:

List resource instances:

```
$ oc get maassubscriptions -n models-as-a-service
$ oc get maasmodelrefs -n <namespace>
$ oc get tenants.maas.opendatahub.io -n models-as-a-service
```

View the YAML configuration of a specific resource:

```
$ oc get maassubscription <subscription-name> -n models-as-a-service -o yaml
$ oc get tenants.maas.opendatahub.io default-tenant -n models-as-a-service -o yaml
```

1.2. PREREQUISITES FOR MODELS-AS-A-SERVICE

Before deploying Models-as-a-Service (MaaS) in Red Hat OpenShift AI, verify that your cluster has the required platform components, operators, and infrastructure resources, and that the necessary configuration flags are enabled.

Platform and access requirements:

- You have a cluster with OpenShift version 4.19.9 or later.
- You have cluster administrator access to install Operators and create cluster-scoped resources.
- Your cluster has a functional ingress controller with valid TLS certificates for external access.
- You have installed OpenShift CLI (**oc**).

Operator requirements:

- You have installed Red Hat OpenShift AI 3.4 or later.
- If you plan to use distributed inference with llm-d, you have completed the setup steps in Enabling distributed inference and configured authentication as described in Enabling authentication and authorization for LLM Inference Service.
- You have installed the Red Hat Connectivity Link Operator version 1.2 or later to the **openshift-operators** namespace and created a **Kuadrant** custom resource in the **kuadrant-system** namespace with ready status. For installation and configuration instructions, see Configuring authentication for llm-d using Red Hat Connectivity Link.
- If you plan to use vLLM runtime with Models-as-a-Service, you have set **spec.dashboardConfig.vLLMDeploymentOnMaaS** to **true** in the **OdhDashboardConfig** custom resource. vLLM runtime for Models-as-a-Service is available as a Technology Preview feature. For more information, see Technology Preview Features Support Scope .

Infrastructure requirements:

- You have created a **DataScienceCluster** resource with the **kserve** component set to **Managed**.
- You have enabled User Workload Monitoring on your OpenShift cluster. User Workload Monitoring is required for MaaS to collect and expose usage metrics for token consumption, request counts, and rate limiting. Without User Workload Monitoring enabled, the MaaS

installation shows a **Degraded** status. For information about enabling User Workload Monitoring, see Enabling monitoring for user-defined projects.

- You have deployed a PostgreSQL database instance, version 14 or later, that is reachable from the OpenShift cluster network. This database is required for API key lifecycle management. OpenShift AI does not provide a PostgreSQL database. You must provision and manage your own PostgreSQL instance.
- You have created a **Secret** named **maas-db-config** in the **redhat-ods-applications** namespace containing the PostgreSQL database connection details. For configuration instructions, see Configure the database secret for Models-as-a-Service.
- You have created a **GatewayClass** resource configured for the OpenShift Gateway Controller (**openshift.io/gateway-controller**) and a **Gateway** named **maas-default-gateway** in the **openshift-ingress** namespace. The **Gateway** resource must include the following annotations:
  - **opendatahub.io/managed: "false"** - Prevents the ODH Model Controller from overriding MaaS-managed authorization policies.
  - **security.opendatahub.io/authorino-tls-bootstrap: "true"** - Enables TLS communication between the **Gateway** and **Authorino**.

  For information about creating Gateway API resources, see Enabling the Gateway API. For example gateway configuration templates, see MaaS gateway configuration examples. These examples are community-maintained and are not supported by Red Hat.
- You have configured TLS for **Authorino** and the MaaS API gateway. For configuration instructions, see Configure TLS for Models-as-a-Service.

NOTE

Deploying large language models might require additional dependencies based on the model size and serving runtime. For comprehensive model serving infrastructure requirements, see Component requirements.

MaaS configuration:

- You have set **spec.components.kserve.modelsAsService.managementState** to **Managed** in the **DataScienceCluster** custom resource.

Dashboard configuration:

- You have set **spec.dashboardConfig.modelAsService** to **true** in the **OdhDashboardConfig** custom resource.

To access MaaS user-facing features in the dashboard:

- You have set **spec.components.llamastackoperator.managementState** to **Managed** in the **DataScienceCluster** custom resource. For more information, see Activating the Llama Stack Operator.
- You have set **spec.dashboardConfig.genAiStudio** to **true** in the **OdhDashboardConfig** custom resource.

To access MaaS administrative features in the dashboard:

- You have set **spec.dashboardConfig.maasAuthPolicies** to **true** in the **OdhDashboardConfig** custom resource.

To enable the MaaS observability dashboard for usage monitoring (optional):

- You have set **spec.dashboardConfig.observabilityDashboard** to **true** in the **OdhDashboardConfig** custom resource. For more information, see Managing observability.
- You have configured observability for OpenShift AI, including metrics storage in the **DSCInitialization** resource and the Cluster Observability Operator. For complete setup instructions, see Managing observability.
- You have enabled observability in the **Kuadrant** custom resource. For more information, see Enable Kuadrant observability for Models-as-a-Service .
- You have enabled telemetry in the **Tenant** custom resource. For more information, see Enable telemetry for Models-as-a-Service.

NOTE

The MaaS observability dashboard is a Technology Preview feature in Red Hat OpenShift AI. For more information about the support scope of Red Hat Technology Preview features, see Technology Preview Features Support Scope .

1.3. CONFIGURE THE DATABASE SECRET FOR MODELS-AS-A-SERVICE

You must create a secret that contains your PostgreSQL database connection details. This database is required for API key lifecycle management in Models-as-a-Service.

Prerequisites

- You have deployed a PostgreSQL database instance, version 14 or later, that is reachable from the OpenShift cluster network. OpenShift AI does not provide a PostgreSQL database. You must provision and manage your own PostgreSQL instance before proceeding.
- You have access to OpenShift CLI (**oc**).
- You have permissions to create secrets in the **redhat-ods-applications** namespace.

Procedure

1. Create the **maas-db-config** secret in the **redhat-ods-applications** namespace:

```
$ oc create secret generic maas-db-config \
  -n redhat-ods-applications \
  --from-literal=DB_CONNECTION_URL=postgresql://<username>:
<password>@<hostname>:<port>/<database>?sslmode=require
```

where:

<username>

Specifies the PostgreSQL database username.

<password>

Specifies the PostgreSQL database password.

<hostname>

Specifies the hostname or IP address of the PostgreSQL server.

<port>

Specifies the port number for the PostgreSQL server, typically **5432**.

<database>

Specifies the name of the PostgreSQL database.
The following example shows a complete connection string:

```
postgresql://maasadmin:XXXXX@pg.example.com:5432/maasdb?sslmode=require
```

2. Optional: Restart the **maas-api** deployment to apply the configuration if **modelsAsService** is already set to **Managed** in the **DataScienceCluster** resource:

```
$ oc rollout restart deployment/maas-api -n redhat-ods-applications
```

This step is not required if the secret exists before you enable **modelsAsService** in the **DataScienceCluster** resource.

Verification

- Verify that the **maas-db-config** secret exists in the **redhat-ods-applications** namespace:

```
$ oc get secret maas-db-config -n redhat-ods-applications
```

Expected output:

```
NAME            TYPE     DATA   AGE
maas-db-config  Opaque   1      5s
```

1.4. CONFIGURE TLS FOR MODELS-AS-A-SERVICE

To enable secure authentication and authorization for model endpoints, you must configure TLS communication between **Authorino** and the Models-as-a-Service (MaaS) API service.

Prerequisites

- You have installed the Red Hat Connectivity Link Operator and created a **Kuadrant** custom resource.
- You have access to OpenShift CLI (**oc**).
- You have cluster administrator privileges for the OpenShift cluster where OpenShift AI is installed.

Procedure

1. Annotate the **Authorino** service to enable service serving certificate generation in OpenShift:

```
$ oc annotate service authorino-authorino-authorization \
  -n kuadrant-system \
  service.beta.openshift.io/serving-cert-secret-name=authorino-server-cert \
  --overwrite
```

The service-ca-operator generates a TLS certificate signed by the cluster service CA and stores it in the **authorino-server-cert** secret.

2. Patch the **Authorino** custom resource to enable the TLS listener:

```
$ oc patch authorino authorino -n kuadrant-system --type=merge --patch '
{
  "spec": {
    "listener": {
      "tls": {
        "enabled": true,
        "certSecretRef": {
          "name": "authorino-server-cert"
        }
      }
    }
  }
}'
```

Authorino uses the generated certificate for inbound TLS communication.

3. Configure the **Authorino** deployment with environment variables for TLS certificate validation:

```
$ oc -n kuadrant-system set env deployment/authorino \
  SSL_CERT_FILE=/etc/ssl/certs/openshift-service-ca/service-ca-bundle.crt \
  REQUESTS_CA_BUNDLE=/etc/ssl/certs/openshift-service-ca/service-ca-bundle.crt
```

The cluster CA bundle is automatically populated by the service-ca-operator in OpenShift.

4. Annotate your **Gateway** resource to enable automatic TLS configuration:

```
$ oc annotate gateway maas-default-gateway \
  -n openshift-ingress \
  security.opendatahub.io/authorino-tls-bootstrap="true" \
  --overwrite
```

The MaaS controller detects this annotation and creates an **EnvoyFilter** resource that configures the Envoy proxy to use TLS when communicating with **Authorino**.

Verification

- Verify that the **Authorino** service has the serving certificate annotation:

```
$ oc get service authorino-authorino-authorization -n kuadrant-system -o
jsonpath='{.metadata.annotations.service.beta.openshift.io/serving-cert-secret-name}'
```

Expected output:

```
authorino-server-cert
```

-

- Verify that the **authorino-server-cert** secret exists:

```
$ oc get secret authorino-server-cert -n kuadrant-system
```

Expected output:

```
NAME                  TYPE                DATA   AGE
authorino-server-cert    kubernetes.io/tls   2      5m
```

- Verify that the **Authorino** CR has TLS enabled:

```
$ oc get authorino authorino -n kuadrant-system -o jsonpath='{.spec.listener.tls.enabled}'
```

Expected output:

```
true
```

- Verify that the **Authorino** deployment has the TLS certificate environment variables configured:

```
$ oc get deployment/authorino -n kuadrant-system -o
jsonpath='{.spec.template.spec.containers[0].env[?(@.name=="SSL_CERT_FILE")].value}'
```

Expected output:

```
/etc/ssl/certs/openshift-service-ca/service-ca-bundle.crt
```

- Verify that the **Gateway** has the TLS bootstrap annotation:

```
$ oc get gateway maas-default-gateway -n openshift-ingress -o
jsonpath='{.metadata.annotations.security.opendatahub.io/authorino-tls-bootstrap}'
```

Expected output:

```
true
```

1.5. VERIFY MODELS-AS-A-SERVICE DEPLOYMENT

After you deploy Models-as-a-Service, you can run a series of checks to confirm that the required custom resources, monitoring components, and tenant configuration are in place.

Prerequisites

- You have deployed Models-as-a-Service.
- You have access to OpenShift CLI (**oc**).
- You have cluster administrator privileges for the OpenShift cluster where OpenShift AI is installed.

Procedure

1. Verify that the Models-as-a-Service (MaaS) custom resource definitions (CRDs) are installed:

```
$ oc get crd | grep maas.opendatahub.io
```

Expected output shows the following CRDs:

```
maasauthpolicies.maas.opendatahub.io
maasmodelrefs.maas.opendatahub.io
maassubscriptions.maas.opendatahub.io
externalmodels.maas.opendatahub.io
tenants.maas.opendatahub.io
```

2. Verify that User Workload Monitoring is enabled on the cluster:

```
$ oc get configmap cluster-monitoring-config -n openshift-monitoring -o
jsonpath='{.data.config.yaml}' | grep enableUserWorkload
```

Expected output:

```
enableUserWorkload: true
```

If User Workload Monitoring is not enabled, the MaaS deployment might show as **Degraded**. For information about enabling User Workload Monitoring, see Enabling monitoring for user-defined projects.

3. Verify that the **Tenant** custom resource exists in the **models-as-a-service** namespace:

```
$ oc get tenants.maas.opendatahub.io -n models-as-a-service
```

Expected output shows at least one **Tenant** resource:

```
NAME            AGE
default-tenant  5m
```

4. Check the status of the **Tenant** custom resource:

```
$ oc get tenants.maas.opendatahub.io default-tenant -n models-as-a-service -o
jsonpath='{.status.conditions[?(@.type=="Ready")].status}'
```

Expected output:

```
True
```

The following values indicate the deployment status:

True

Indicates that the MaaS deployment is successful and all prerequisites are met.

False or **Degraded**

Indicates missing prerequisites or configuration issues. Check the condition message for details:

```
$ oc get tenants.maas.opendatahub.io default-tenant -n models-as-a-service -o
jsonpath='{.status.conditions[?(@.type=="Ready")].message}'
```

5. Verify that the **Tenant** custom resource is configured:

```
$ oc get tenants.maas.opendatahub.io default-tenant -n models-as-a-service
```

Expected output:

```
NAME             READY   REASON
default-tenant   True    AllComponentsReady
```

6. Optional: If you plan to use external models, verify that the **ExternalModel** CRD is available:

```
$ oc get crd externalmodels.maas.opendatahub.io
```

Expected output shows the CRD details:

```
NAME                                  CREATED AT
externalmodels.maas.opendatahub.io    2026-04-28T10:15:30Z
```

Verification

- All MaaS CRDs are deployed and available.
- User Workload Monitoring is enabled on the cluster.
- The **Tenant** custom resource shows a **Ready** status with reason **AllComponentsReady**.

Troubleshooting

If the **Tenant** status shows **False** or **Degraded**:

- Verify that User Workload Monitoring is enabled on the cluster.
- Verify that all prerequisites in Prerequisites for Models-as-a-Service are met.
- Check that the PostgreSQL database secret **maas-db-config** exists in the **redhat-ods-applications** namespace.
- Verify that Red Hat Connectivity Link is installed and the **Kuadrant** custom resource is ready.
- Review the **Tenant** condition messages for specific error details.

1.6. PUBLISH MODELS WITH MODELS-AS-A-SERVICE

In Red Hat OpenShift AI, you can deploy generative AI models through the dashboard wizard and publish them to Models-as-a-Service (MaaS) so that administrators can enforce subscription-based quota and token limits.

Prerequisites

- You are logged in to the OpenShift AI dashboard.
- You have administrator access to a project in OpenShift AI.
- Your cluster administrator has installed the required operators and infrastructure for Models-as-a-Service. For more information, see Prerequisites for Models-as-a-Service.
- If you plan to use distributed inference with llm-d, your cluster administrator has completed the setup steps in Enabling distributed inference and configured authentication as described in Enabling authentication and authorization for LLM Inference Service .
- If you plan to use vLLM runtime with Models-as-a-Service, your cluster administrator has enabled the vLLM deployment on MaaS feature flag: **spec.dashboardConfig.vLLMDeploymentOnMaaS: true** in the **OdhDashboardConfig** custom resource. vLLM runtime support for Models-as-a-Service is a Technology Preview feature.

Procedure

1. In the left navigation menu, click **Projects**.
2. Click the name of the project where you want to deploy the model.
3. Click the **Deployments** tab.
4. Click **Deploy model** to open the wizard.
5. In the **Model details** section, specify your storage connection and model path.
6. Complete the model configuration based on whether **vLLMDeploymentOnMaaS** is enabled in the **OdhDashboardConfig** custom resource:

   If vLLMDeploymentOnMaaS is enabled

   a. In the **Model details** section, select **Generative AI model (Example, LLM)** as the model type.

   b. Make sure that the **Use legacy deployment method** checkbox is unchecked.

   c. Click **Next**.

   d. In the **Model deployment** section, enter a unique model deployment name using lowercase letters, numbers, and hyphens.

   e. Select an appropriate hardware profile for your model.

   f. Select one of the following deployment resources:

   - **Distributed inference with llm-d** for distributed inference support.
   - A vLLM-based **LLMInferenceServiceConfig**, such as **vLLM NVIDIA CUDA GPU LLMInferenceServiceConfig**, for vLLM-based serving as a Technology Preview feature in Red Hat OpenShift AI.

   If vLLMDeploymentOnMaaS is not enabled

a. In the **Model details** section, select **Generative AI model (Example, LLM)** as the model type.

b. Click **Next**.

c. In the **Model deployment** section, enter a unique model deployment name using lowercase letters, numbers, and hyphens.

d. Select an appropriate hardware profile for your model.

e. Select **Distributed inference with llm-d** as the deployment resource.

NOTE

Models deployed for MaaS use the **LLMInferenceService** architecture, which is designed for large language models and integrates with the MaaS gateway for subscription-based quota enforcement. The legacy deployment method uses traditional KServe **InferenceService** resources with serving runtimes.

7. In the **Model deployment** section, in the **Number of replicas** field, enter the number of replicas to deploy. The default is 1. For production workloads, consider deploying at least 2 replicas for high availability.

8. Click **Next**.

9. In the **Advanced settings** section, configure MaaS publishing:

   a. Under **Model availability**, select **Publish as MaaS** to make the model accessible to users through the MaaS gateway.

NOTE

Publishing as MaaS creates a **MaaSModelRef** object that registers the model with MaaS for subscription assignment. After publishing, an administrator must create a subscription and add this model to make it accessible to user groups.

   b. Optional: Select **Add custom runtime arguments** or **Add custom runtime environment variables** to customize model behavior.

   c. Click **Next**.

10. In the **Review** section, verify your configuration settings:

    a. Review the model details, deployment configuration, and advanced settings.

    b. Click **Deploy model**.

Verification

- Verify that the model appears on the **Deployments** tab with a checkmark in the **Status** column.
- Verify that the model was published to MaaS:

```
$ oc get maasmodelref -n <your-project-namespace>
```

You should see a **MaaSModelRef** object for your deployed model.

NEXT STEPS

Models published to MaaS require subscription and authorization policy configuration before users can access them.

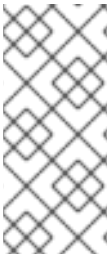
- Create a subscription
- Create an authorization policy

Additional resources

- Models-as-a-Service administration troubleshooting

1.7. MODELS-AS-A-SERVICE SUBSCRIPTIONS

In Red Hat OpenShift AI, you can use Models-as-a-Service (MaaS) subscriptions to manage quotas and token limits for AI model serving. With subscriptions, you can grant specific groups quotas for models with configurable token limits based on user group membership.



NOTE

In OpenShift AI 3.3, MaaS used a tier-based model for access control. Starting with OpenShift AI 3.4, tiers have been replaced with subscriptions. The subscription model provides more flexibility by allowing users to belong to multiple subscriptions and uses custom resource definition (CRD)-based configuration for improved GitOps compatibility.

1.7.1. Subscription-based access control

When multiple teams share large language models, you can use subscriptions to perform the following tasks:

- Prevent resource exhaustion by enforcing token limits per model
- Provide different access levels for different user groups
- Track and allocate costs based on team consumption
- Control which teams can access high-cost or sensitive models
- Allow users to belong to multiple subscriptions based on their group memberships

MaaS assigns users to subscriptions based on their OpenShift group membership. When a user belongs to multiple groups with different subscriptions, the system uses the subscription with the highest priority level.

1.7.2. Subscription properties

MaaS subscriptions are defined as **MaaSSubscription** custom resources in the cluster. Each subscription has the following properties:

Name

A unique identifier for the subscription that becomes the Kubernetes resource name.

Description

An optional human-readable description shown in the dashboard.

Groups

One or more groups whose members can access this subscription. Groups can come from OpenShift Group objects or external OIDC providers. Users can belong to multiple groups and therefore have access to multiple subscriptions.

Priority level

A numeric value that determines subscription precedence when creating an API key without specifying a subscription. Higher numbers indicate higher priority, with 0 as the lowest. Priority only applies during API key creation.

Models

A list of models that this subscription gives quota for, with configurable token limits for each model.

1.7.3. Priority levels

Priority levels determine which subscription is selected when creating an API key without explicitly specifying a subscription. When a user belongs to multiple groups with different subscriptions, the subscription with the highest priority is selected as the default.

For example, if a user belongs to both the **analytics-team** group with priority 1 and the **production-apps** group with priority 2, creating a key without specifying a subscription selects the **production-apps** subscription because it has the higher priority.

When creating API keys, specifying the subscription explicitly bypasses priority selection.

1.7.3.1. Priority level recommendations

Use a consistent priority numbering scheme to make subscription precedence clear and maintainable.

Recommended priority scheme:

Production workloads: 100

Use this priority for customer-facing applications, production APIs, and critical business processes.

Staging and pre-production: 50

Use this priority for QA testing, user acceptance testing, and performance testing.

Development and experimentation: 0

Use this priority for exploratory data science work, prototype development, and learning. This is the default value.

Personal and sandbox: -10

Use this priority for individual experimentation, tutorials, and non-business use.

Common use cases:

Separate production and development resources

Create a production subscription with priority 100 for stricter quotas billed to production cost

centers, and a development subscription with priority 0 for generous quotas billed to R&D. Production applications automatically use the production subscription, while developers must explicitly select the development subscription for testing.

Team-based access with overlapping membership

When users belong to multiple teams, assign higher priority to broader access. For example, set "ML Platform Team" to priority 10 for access to all models and "Analytics Team" to priority 5 for analytics-focused models. Users in both teams default to the ML Platform Team subscription.

Cost-tiered model access

Create a "Standard Models" subscription with priority 10 for cheaper models with higher quotas, and a "Premium Models" subscription with priority 0 for expensive models with limited quotas. Users consume cheaper resources by default and must explicitly select the premium subscription when needed.

Configuration guidance:

- Use incremental priority values with reasonable gaps such as 10, 20, 30 rather than 0, 1000, 10000. This makes it easier to insert intermediate priorities later.
- Avoid setting all subscriptions to the same priority, which creates unpredictable behavior.
- Use priority for convenience and defaults, not for access control. Use authorization policies to restrict which users can access which models.

1.7.4. Token limits

Tokens are the basic units of text processing in large language models. Token limits control the maximum number of tokens that can be consumed per request or time period for a specific model.

Configure token limits for each model when you create or edit a subscription through the dashboard.

Each model in a subscription can have different token limit configurations, allowing administrators to provide varying levels of access to different models within the same subscription.

1.7.5. Relationship with authorization policies

Subscriptions and authorization policies work together to control model access in the following ways:

- **Subscriptions** give groups quota for specific models with token rate limits.
- **Authorization policies** grant groups access to model endpoints through the API gateway.

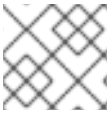
Both are required for users to access models through MaaS. A subscription defines quota limits for model usage, while an authorization policy enables API gateway access.

When you create a subscription, you can optionally create a matching authorization policy by selecting the **Create matching authorization policy**checkbox. The authorization policy uses the same groups and models as the subscription, so users can access the models as soon as the subscription is created.

For more information about authorization policies, see Manage Models-as-a-Service authorization policies.

1.8. MANAGE MODELS-AS-A-SERVICE SUBSCRIPTIONS

You can create and manage Models-as-a-Service (MaaS) subscriptions to control group access to models and configure token limits.



NOTE

In OpenShift AI 3.4, MaaS uses subscriptions instead of tiers.

1.8.1. View subscriptions

In Red Hat OpenShift AI, you can use the **Subscriptions** page to monitor service subscriptions, verify their status, and review associated models and priority levels.

Prerequisites

- You are logged in to the OpenShift AI dashboard.
- You have administrator access to the OpenShift AI dashboard.

Procedure

1. In the OpenShift AI dashboard, click **Settings → Subscriptions**.
2. Review the information in the Subscriptions table:

   Name
   The unique identifier for the subscription.

   Phase
   The current status of the subscription. Possible values: **Active**, **Failed**.

   Groups
   The number of OCP or custom user groups assigned to the subscription.

   Models
   The count of Models-as-a-Service (MaaS) model references included in the subscription.

   Priority
   The priority level assigned to the subscription. Higher numbers indicate higher priority.

3. Optional: To filter and organize the view:
   - Use the **Keyword** dropdown to select a filter criterion.
   - Enter text in the **Filter by name or description** field to search for specific subscriptions.
   - Click the column headers to sort by Name, Phase, Groups, Models, or Priority.
4. Click a subscription name to open the details pane.
5. In the details view, review the subscription configuration including groups, models, token limits, and synchronization status.

Verification

- Verify that the Subscriptions page displays all configured subscriptions in the system.
- Verify that subscriptions show the correct phase status: **Active** or **Failed**.

- Click a subscription name and verify that you can view its complete configuration.

1.8.2. Create a subscription for Models-as-a-Service

In Red Hat OpenShift AI, you can create a Models-as-a-Service (MaaS) subscription to grant user groups quota for specific models with configurable token limits.

Prerequisites

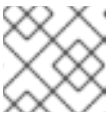
- You are logged in to the OpenShift AI dashboard.
- You have administrator access to the OpenShift AI dashboard.
- You have published at least one model to Models-as-a-Service (MaaS).

Procedure

1. In the OpenShift AI dashboard, click **Settings → Subscriptions**.
2. Click **Create subscription**.
3. In the **Create subscription** form, configure the subscription:
    a. **Name**: Enter a descriptive name for the subscription.
    b. Optional: Click **Edit resource name** to set a custom internal identifier for the subscription. If not specified, the resource name is generated automatically from the display name.
    c. Optional: **Description**: Provide a brief description of the purpose of the subscription.
    d. **Priority**: Set the subscription priority level using a numeric value. Higher numbers indicate higher priority. When a user belongs to multiple groups with different subscriptions, the subscription with the highest priority level is used.
    e. **Groups**: Select OpenShift groups or enter custom group names from your external OIDC provider. Users who are members of these groups can access the models included in this subscription.
4. Configure models and token limits:
    a. Click **Add models**.
    b. In the **Add models** dialog, review the available models and their associated projects, model IDs, existing subscriptions, and policies.
    c. Click **Add model** for each model you want to include in this subscription.
    d. Click **Add models** to confirm your selection.
    e. For each model in the subscription, click **Add token limit** to configure token consumption limits.
    f. In the **Edit subscription token limits** dialog, enter the number of tokens allowed.
    g. Enter the time period value.
    h. Select the time unit: hour, minute, or second.

i. Optional: Click **Add token rate limit** to configure additional token limits with different time windows.

j. Click **Save**.



NOTE

At least one token limit is required for each model in the subscription.

5. Optional: Select **Create a matching authorization policy** to automatically create an authorization policy for this subscription.

NOTE

To consume model endpoints through the API gateway, users must have both a subscription and an authorization policy. A subscription defines quota for models. An authorization policy is a separate resource that authorizes specific groups to access model endpoints through the API gateway. While you can automatically create a matching authorization policy during subscription creation, creating authorization policies separately provides more flexibility and control.

IMPORTANT

The matching authorization policy feature creates an initial authorization policy with the same groups and models as the subscription. However, changes made to the subscription later such as adding or removing groups or models require manual updates to the authorization policy. When you modify a subscription, you must manually update the corresponding authorization policy to keep them synchronized.

6. Click **Create subscription**.

Verification

- In the OpenShift AI dashboard, navigate to **Settings → Subscriptions** and verify that the new subscription appears in the list with a phase status of **Active**.
- Click the subscription name to view its details and confirm the groups, models, token limits, and priority level are configured correctly.

Next steps

- If you did not select **Create a matching authorization policy**, Create an authorization policy
- Create an API key for a user

1.8.3. Edit a subscription

In Red Hat OpenShift AI, you can edit a Models-as-a-Service (MaaS) subscription to add or remove models, change token limits, or update group assignments. Changes modify the **MaaSSubscription** custom resource.

Prerequisites

- You are logged in to the OpenShift AI dashboard.
- You have administrator access to the OpenShift AI dashboard.
- At least one MaaS subscription exists.

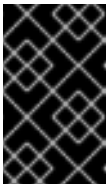
IMPORTANT

Editing a **MaaSSubscription** resource takes effect when you save the changes:

- Changing token limits affects all users in the subscription groups.
- Adding or removing groups grants or revokes subscription quota for those groups.
- Adding or removing models changes which models are available to users in the subscription groups.

Procedure

1. In the OpenShift AI dashboard, click **Settings → Subscriptions**.
2. Locate the subscription you want to modify and click its name.
3. Click the action menu (⋮), and then select **Edit**.
4. Modify the subscription properties:
   a. Update the name or description if needed.
   b. Adjust the priority level.
   c. Add or remove groups.
   d. Add or remove models by clicking **Add models** or using the action menu (⋮) next to each model.
   e. Modify token limits for existing models by clicking **Add token limit** or editing existing limits.



IMPORTANT

The **metadata.name** of the **MaaSSubscription** custom resource cannot be changed after creation. If you need a different resource name, delete the subscription and create a new one.

5. Click **Save**.

IMPORTANT

If you created a matching authorization policy when you created this subscription, changes to the subscription such as adding or removing groups or models require manual updates to the authorization policy. After modifying a subscription, manually update the corresponding authorization policy to keep them synchronized.

Verification

- In the OpenShift AI dashboard, navigate to **Settings → Subscriptions**.
- Verify that the subscription shows the updated configuration.
- Optional: Check the **MaaSSubscription** custom resource to confirm the changes:

```
$ oc get maassubscription <subscription-name> -n models-as-a-service -o yaml
```

1.8.4. Delete a subscription

In Red Hat OpenShift AI, you can delete a Models-as-a-Service (MaaS) subscription to revoke quota for specific user groups. Deleting a subscription removes the quota limits for all groups included in that subscription.

Prerequisites

- You have access to the OpenShift AI dashboard with administrator privileges.
- You have created at least one MaaS subscription.

Deleting a subscription revokes quota for all groups included in that subscription. Make sure that affected users have access to models through other subscriptions before deletion to avoid service disruption.

Procedure

1. In the OpenShift AI dashboard, click **Settings → Subscriptions**.
2. Locate the subscription you want to delete and click its name.
3. Click the action menu (⋮), and then select **Delete**.
4. In the **Delete subscription** dialog, review the impact of deletion:
   - Users in groups assigned to this subscription lose access to the included models.
   - All API keys bound to this subscription are invalidated.
   - Authorization policies are not deleted automatically. After deletion, review and update any authorization policies that referenced this subscription's groups or models.
5. To confirm, type the subscription name and click **Delete**.

Verification

- In the OpenShift AI dashboard, navigate to **Settings → Subscriptions**.
- Verify that the deleted subscription no longer appears in the list.
- Optional: List the remaining subscriptions to confirm the expected state:

```
$ oc get maassubscription -n models-as-a-service
```

Next steps

Next steps

- View authorization policies to review and update policies that referenced groups or models in the deleted subscription.

Additional resources

- Models-as-a-Service administration troubleshooting

1.9. MANAGE MODELS-AS-A-SERVICE AUTHORIZATION POLICIES

You can create and manage authorization policies to control which groups can access AI model endpoints through the API gateway.

1.9.1. Models-as-a-Service authorization policies

In Red Hat OpenShift AI, you can use Models-as-a-Service (MaaS) authorization policies in combination with subscriptions to control user access to model endpoints through the API gateway.

A subscription gives groups quota for specific models with token rate limits. An authorization policy is required to authorize groups to access model endpoints through the API gateway.

IMPORTANT

Both a subscription and an authorization policy are required for users to access models through Models-as-a-Service:

- **Subscription**: Defines quota for models with token rate limits.
- **Authorization policy**: Authorizes groups to access model endpoints through the API gateway.

Without an authorization policy, users receive 403 Forbidden errors even if they have a valid subscription.

An authorization policy consists of the following components:

Name
: A unique identifier for the policy.

Description
: An optional description explaining the purpose of the policy.

Groups
: The groups authorized to access model endpoints through this policy. Groups can come from OpenShift Group objects or external OIDC providers depending on the authentication method.

Models
: The model endpoints that authorized groups can access.

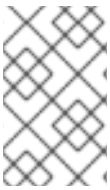
Phase
: The current status of the policy: **Active**, **Failed**, or **Unknown**.

When you create a subscription, you can optionally create a matching authorization policy automatically by selecting the **Create matching authorization policy** checkbox. This creates an authorization policy with the same groups and models as the subscription, ensuring that users can immediately access the

models included in their subscription.

Authorization policy lifecycle

- **Creating policies**: Create authorization policies manually or automatically when creating a subscription.
- **Active phase**: When a policy is in **Active** phase, the specified groups can access the configured model endpoints through the API gateway.
- **Failed phase**: If a policy enters **Failed** phase, check the policy status conditions for error messages.
- **Updating policies**: You can add or remove groups and models from existing policies. Changes take effect immediately.
- **Deleting policies**: Deleting an authorization policy immediately revokes API gateway access for all groups in that policy.



NOTE

Authorization policies are managed as Models-as-a-Service (MaaS) custom resources in OpenShift. You can manage them through the OpenShift AI dashboard or using OpenShift CLI (**oc**).

1.9.2. View authorization policies

In Red Hat OpenShift AI, you can view all Models-as-a-Service (MaaS) authorization policies in the dashboard to see which groups can access specific model endpoints.

Prerequisites

- You have access to the OpenShift AI dashboard with administrator privileges.
- At least one MaaS authorization policy exists.

Procedure

1. In the OpenShift AI dashboard, click **Settings → Authorization policies**.
   The **Authorization policies** page displays a table with the following columns:
   - **Name**: The name of the authorization policy. Policies created automatically from subscriptions include the subscription name in the policy name.
   - **Phase**: The current status of the policy. Possible values: **Active**, **Failed**, **Unknown**.
   - **Groups**: The number of OpenShift groups authorized by this policy.
   - **Models**: The number of model endpoints included in this policy.
2. Optional: Filter the list of authorization policies:
   - To filter by keyword, click the **Keyword** dropdown and select a filter option.
   - To search by name or description, enter text in the search field.

3. Optional: Sort the table by clicking any column header.
4. To view details of a specific authorization policy:
   a. Click the action menu (⋮) for the policy you want to view.
   b. Select **View details**.
   The policy details page displays:
      - Policy name, description, phase, and resource name
      - Creation date
      - Groups authorized by this policy
      - Models included in this policy, along with the namespace each model is deployed in

Verification

- The Authorization policies table displays all policies with their current status.
- When you view a policy's details, the Groups and Models sections show the configuration for that policy.

1.9.3. Create an authorization policy

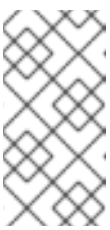
In Red Hat OpenShift AI, you can create authorization policies to control which groups can access AI model endpoints through the API gateway.

Prerequisites

- You have access to the OpenShift AI dashboard with administrator privileges.
- You have published at least one model to Models-as-a-Service (MaaS).

Procedure

1. In the OpenShift AI dashboard, click **Settings → Authorization policies**.
2. Click **Create authorization policy**.
3. In the **Create authorization policy** dialog, configure the following settings:
   a. In the **Name** field, enter a unique name for the authorization policy.



NOTE

By default, the resource name matches the policy name. To customize the resource name, click **Edit resource name** and enter a different value. The resource name identifies the underlying **MaaSAuthPolicy** custom resource in OpenShift.

   b. Optional: In the **Description** field, enter a description explaining the purpose of the policy.
   c. From the **Groups** dropdown, select the groups to authorize for API gateway access.

You can select multiple groups or type to add a new group name. Groups can come from OpenShift Group objects, API key group snapshots, or OIDC token claims depending on how users authenticate.

d. In the **Models** section, click **Add models**.

e. In the **Add models to authorization policy** dialog, browse the list of available models or use the search field to filter by name or description.

f. Review the **Subscriptions** and **Policies** columns to see which subscriptions and policies already include each model.

g. For each model you want to add, click **Add model**.

h. Click **Add models** to add the selected models to the policy.
The **Models** section displays a table with the selected models and their project namespaces.

4. Click **Create authorization policy**.

NOTE

If you create a subscription with the **Create matching authorization policy** option selected, an authorization policy is created automatically with the same groups and models as the subscription. You only need to create authorization policies manually when you want to configure API gateway access independently of subscriptions.

Verification

- The new authorization policy appears in the Authorization policies table with an **Active** phase.

Next steps

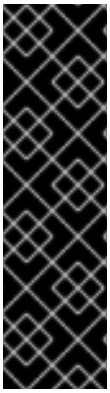
- Create an API key for a user

1.9.4. Edit an authorization policy

In Red Hat OpenShift AI, you can use the dashboard to edit Models-as-a-Service (MaaS) authorization policies to add or remove authorized groups and model endpoints.

Prerequisites

- You have access to the OpenShift AI dashboard with administrator privileges.
- At least one authorization policy exists.



IMPORTANT

Editing an authorization policy takes effect when you save your changes:

- Adding groups grants API gateway access to users in those groups.
- Removing groups revokes API gateway access for users in those groups, including users with active sessions.
- Users and applications might experience access changes or receive 403 Forbidden errors based on the new configuration.

Procedure

1. In the OpenShift AI dashboard, click **Settings → Authorization policies**.
2. In the row for the authorization policy you want to edit, click the action menu (⋮), and then select **Edit**.
3. In the **Edit authorization policy** dialog, modify the following settings as needed:
   a. Update the **Name** field to change the policy display name.
   b. Update the **Description** field to change the policy description.
   c. From the **Groups** dropdown, add or remove groups:
      - To add groups, select additional groups from the dropdown or type to add a new group name.
      - To remove a group, click the remove icon (×) next to the group name.

NOTE

The available groups depend on your authentication configuration: OpenShift groups when using OpenShift authentication, or OIDC group claims when using external OIDC.

   d. In the **Models** section, add or remove models:
      - To add models, click **Add models** to open the **Add models to authorization policy** dialog. Select the models to add, and then click **Add models** in the dialog to confirm.
      - To remove a model, click the action menu (⋮) for the model, and then select **Remove**.
4. Click **Save** to apply your changes.

Verification

- The updated authorization policy appears in the Authorization policies table. To confirm specific changes, click the policy name to open the detail view.
- Confirm the updated configuration on the **MaaSAuthPolicy** resource:

```
$ oc get maasauthpolicy <policy-name> -n models-as-a-service -o yaml
```

1.9.5. Delete a Models-as-a-Service authorization policy

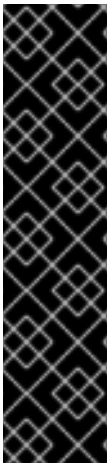
In Red Hat OpenShift AI, you can delete Models-as-a-Service (MaaS) authorization policies to revoke API gateway access for groups included in the policy. If another authorization policy grants the same groups access to the same models, those groups retain access through the remaining policy.

Prerequisites

- You have access to the OpenShift AI dashboard with administrator privileges.
- At least one **MaaSAuthPolicy** resource exists.

Procedure

1. In the OpenShift AI dashboard, click **Settings → Authorization policies**.
2. In the row for the authorization policy you want to delete, click the action menu (⋮), and then select **Delete**.
3. In the **Delete policy** dialog, enter the authorization policy name to confirm deletion.
4. Click **Delete**.



IMPORTANT

Deleting an authorization policy revokes API gateway access for all groups in that policy. Users and applications using models covered by this policy receive 403 Forbidden errors even if they have a valid subscription.

If you delete an authorization policy that was automatically created with a subscription, the subscription remains active and continues to enforce token limits. API gateway access requires a new authorization policy.

When removing a user from a group, you must manually revoke all associated API keys to immediately revoke access. Consider setting up automation to revoke API keys when users are removed from groups.

Verification

- The deleted authorization policy no longer appears in the Authorization policies table.
- Confirm the **MaaSAuthPolicy** resource was deleted:

```
$ oc get maasauthpolicy <policy-name> -n models-as-a-service
```

Expected output:

```
Error from server (NotFound): maasauthpolicies.maas.opendatahub.io "<policy-name>" not found
```

Next steps

- Create an authorization policy

Additional resources

- Subscriptions
- Models-as-a-Service administration troubleshooting

1.10. MANAGE API KEYS FOR USERS

You can create and manage API keys on behalf of users to provide them with programmatic access to models through Models-as-a-Service subscriptions.

1.10.1. View API keys

In Red Hat OpenShift AI, you can view and filter all API keys created by users in your organization to monitor key usage and identify keys that might need to be revoked.

Prerequisites

- You have access to the OpenShift AI dashboard with administrator privileges.

Procedure

1. In the OpenShift AI dashboard, click **Gen AI studio → API keys**.
   The **API keys** page displays a table with the following columns:
   - **Name**: The name assigned to the API key
   - **Status**: The current state of the key. Possible values: **Active**, **Expired**, **Revoked**.
   - **Subscription**: The subscription that scopes the key's access to models
   - **Owner**: The username of the key owner
   - **Created**: The date when the key was created
   - **Last used**: The date when the key was last used to access a model
   - **Expires**: The expiration date for the key
2. Optional: Filter the list of API keys:
   - To filter by status, click the **Status** dropdown and select **Active**, **Expired**, or **Revoked**.
   - To filter by username, enter a username in the search field.
3. Optional: Sort the table by clicking any column header.
4. Optional: If the list contains more keys than fit on one page, use the pagination controls at the bottom of the table to navigate between pages.

Verification

- The **API keys** table lists at least one key with a **Status** value of **Active**, **Expired**, or **Revoked**.
- Each key displays information including owner, subscription, creation date, last used date, and expiration.

1.10.2. Create an API key for a user

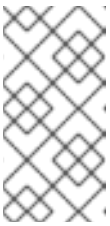
In Red Hat OpenShift AI, you can create API keys on behalf of users to provide them with programmatic access to models through Models-as-a-Service subscriptions.

Prerequisites

- You have access to the OpenShift AI dashboard with administrator privileges.
- You have created at least one Models-as-a-Service (MaaS) subscription.

Procedure

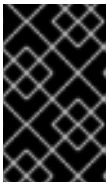
1. In the OpenShift AI dashboard, click **Gen AI studio → API keys**.
2. Click **Create API key**.
3. In the **Create API key** dialog, configure the following settings:
   a. In the **Name** field, enter a descriptive name for the API key.
   b. Optional: In the **Description** field, enter additional details about what the key is for.
   c. From the **Subscription** dropdown, select the subscription that determines which models the key can access and the applicable token limits.
   The **Models** section displays the models included in the selected subscription and their configured token limits.
   d. From the **Expiration** dropdown, select the number of days until the key expires.



NOTE

You can select an expiration period from 1 to 365 days. The default expiration is 30 days. As an administrator, you can set a maximum expiration limit in the **Tenant** custom resource. If not set, the default maximum is 90 days.

4. Click **Create**.
   The **API key created** dialog displays the generated key with a prefix of **sk-oai-**.



IMPORTANT

The plaintext key is displayed only during creation and cannot be retrieved later. Save the key in a secrets manager before closing the **API key created** dialog. If you lose the key, you must revoke it and create a new one.

5. Click the copy icon next to the API key field to copy the key, and then provide it to the user through a secure channel.
6. Click **Close**.

Verification

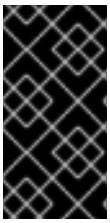
- The new API key appears in the API keys table with an **Active** status.

Next steps

- Make API calls to models

1.10.3. Revoke user API keys

As an administrator, you can revoke individual Models-as-a-Service (MaaS) API keys, or all the keys belonging to a specific user.



IMPORTANT

API keys retain a snapshot of user group memberships from when the key was created. If a user is removed from a group, their existing API keys continue to grant access until you revoke them. To immediately revoke access after a group change, revoke the API keys for that user.

Prerequisites

- You have administrator privileges for OpenShift AI.
- You have access to the OpenShift AI dashboard with administrator privileges.
- At least one API key exists.

Procedure

To revoke an individual API key:

1. In the OpenShift AI dashboard, click **Gen AI studio → API keys**.
2. In the row for the API key you want to revoke, click the action menu (⋮) and select **Revoke**.
3. In the **Revoke API key?** dialog, enter the API key name to confirm revocation.
4. Click **Revoke**.

To revoke all API keys for a single user:

1. In the OpenShift AI dashboard, click **Gen AI studio → API keys**.
2. Click the action menu (⋮) in the table header and select **Revoke user API keys**
3. In the **Revoke user API keys** dialog, enter the username in the **Username** field.
4. Click the search icon to display the keys for that user.
5. Click **Revoke all keys** to revoke all API keys for the user.

Verification

- Verify that the revoked API key or keys display a **Revoked** status in the API keys table.
- Verify that the revoked API key cannot make API calls:

```
$ curl -H "Authorization: Bearer <revoked-api-key>" \
  https://<maas-gateway-url>/maas-api/v1/models
```

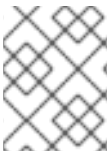
Expected output: **401 Unauthorized** error indicating the key is no longer valid.

1.10.4. Configure the API key expiration limit

In Red Hat OpenShift AI, you can set a maximum expiration period for API keys to enforce security policies and prevent users from creating keys with extended expiration periods.

Prerequisites

- You have cluster administrator privileges for the OpenShift cluster where OpenShift AI is installed.
- You have deployed Models-as-a-Service.
- The **Tenant** custom resource exists in the **models-as-a-service** namespace.



NOTE

Changes to **maxExpirationDays** apply only to API keys created after the change. Existing keys retain their original expiration dates.

Procedure

- Edit the **Tenant** custom resource to configure the API key expiration limit, using one of the following methods:

  Using the OpenShift console

  a. In the OpenShift console, navigate to **Administration → CustomResourceDefinitions**.

  b. Search for **Tenant** and click the resource name.

  c. Click the **Instances** tab.

  d. Click **default-tenant**.

  e. Click the **YAML** tab.

  f. In the YAML editor, locate the **spec** section and add or update the **apiKeys** configuration:

  ```
  spec:
    apiKeys:
      maxExpirationDays: 90
  ```

  The YAML file uses the following field:

  maxExpirationDays

  Specifies the maximum allowed expiration in days for API keys. Common values: 30 for one month, 90 for three months, or 365 for one year. If not set, the default is 90 days.

  a. Click **Save**.

  Using the OpenShift CLI

Run the following command:

```
$ oc patch tenants.maas.opendatahub.io default-tenant -n models-as-a-service \
  --type merge \
  -p '{"spec":{"apiKeys":{"maxExpirationDays":90}}}'
```

Verification

- Confirm that the **maxExpirationDays** value is set on the **Tenant** resource:

  ```
  $ oc get tenants.maas.opendatahub.io default-tenant -n models-as-a-service -o
  jsonpath='{.spec.apiKeys.maxExpirationDays}'
  ```

  The output is the value you configured, for example, **90**.

- Optional: Test that the limit is enforced by attempting to create an API key with an expiration period that exceeds the configured maximum. The request receives a 400 Bad Request response with an error message indicating the expiration exceeds the configured maximum.

Additional resources

- Subscriptions
- Models-as-a-Service administration troubleshooting

1.11. MANAGE MODELS-AS-A-SERVICE USING THE CLI AND API

In Red Hat OpenShift AI, you can manage Models-as-a-Service (MaaS) configurations by using the MaaS API and custom resources. This approach is useful for the following scenarios:

- External OpenID Connect (OIDC) users who cannot access the dashboard
- Automating repetitive configuration tasks
- Integrating MaaS management into GitOps workflows and CI/CD pipelines

1.11.1. Models-as-a-Service API overview

In Red Hat OpenShift AI, you can use the Models-as-a-Service (MaaS) API to manage subscriptions, authorization policies, and API keys, and to call model endpoints programmatically. You can access the API directly through HTTP clients such as **curl** or integrate it with automation tools and GitOps workflows.

1.11.1.1. API structure

MaaS provides two APIs:

Management API

The MaaS management API (/**maas-api/v1**) provides endpoints for management operations such as creating API keys, listing models, and managing user access. This API accepts both OIDC tokens and API keys for authentication.

Inference API

Model-specific inference endpoints (**/llm/<model-name>/v1**) use OpenAI-compatible request and response formats for model interactions. These endpoints require API key authentication.

1.11.1.2. Authentication methods

The MaaS API supports two authentication methods:

OIDC tokens

Users authenticated through an external OIDC provider authenticate with JWTs obtained from their identity provider. OIDC tokens are required for management API operations that use external OIDC authentication.

API keys

Users authenticate all management API operations and all inference operations by using API keys with the **sk-oai-** prefix. API keys can be created through the MaaS API or, for OpenShift-authenticated users, through the dashboard.

Additional resources

- Models-as-a-Service API endpoints reference

1.11.2. MaaS custom resource workflow

To make a model available to your users through Models-as-a-Service (MaaS), you can create and apply custom resources by using YAML and OpenShift CLI (**oc**) in the following order:

- Publish a model by creating a **MaaSModelRef** resource that references your deployed inference server.
- Create a subscription by defining a **MaaSSubscription** resource that assigns groups or users to the published model with token quota.
- Create an authorization policy by defining a **MaaSAuthPolicy** resource that grants API gateway access to the same groups or users.

For a complete list of MaaS custom resources, see Models-as-a-Service custom resources.

1.11.3. Publish a model to Models-as-a-Service using YAML

You can publish a deployed model to Models-as-a-Service (MaaS) by creating a **MaaSModelRef** custom resource. After you publish the model, administrators can add it to MaaS subscriptions and authorization policies.

Prerequisites

- You have cluster administrator privileges for the OpenShift cluster where OpenShift AI is installed.
- You have access to OpenShift CLI (**oc**).
- You have deployed Models-as-a-Service.
- You have deployed a model using one of the supported serving methods:
  - LLM-D distributed inference

- vLLM runtime
- External LLM provider

Procedure

1. Create a YAML file named, for example, **maasmodelref.yaml**, defining the **MaaSModelRef** custom resource:

```
apiVersion: models.opendatahub.io/v1alpha1
kind: MaaSModelRef
metadata:
  name: <model-ref-name>
  namespace: <model-namespace>
spec:
  modelRef:
    kind: <backend-kind>
    name: <backend-resource-name>
```

where:

metadata.name

Specifies a unique resource name for the model reference.

metadata.namespace

Specifies the namespace where the model reference is created. This must be the same namespace as the backend resource.

spec.modelRef.kind

Specifies the backend resource type. Allowed values: **LLMInferenceService**, **ExternalModel**.

spec.modelRef.name

Specifies the name of the backend resource (for example, an **LLMInferenceService** or **ExternalModel** resource).

NOTE

For models served using the vLLM runtime or LLM-D distributed inference, use **kind: LLMInferenceService**. For models hosted by external providers such as OpenAI or Anthropic, use **kind: ExternalModel**.

Example configuration for vLLM model

```
apiVersion: models.opendatahub.io/v1alpha1
kind: MaaSModelRef
metadata:
  name: llama-3-8b-instruct
  namespace: model-serving
spec:
  modelRef:
    kind: LLMInferenceService
    name: llama-3-8b-instruct
```

Example configuration for external model

```
apiVersion: models.opendatahub.io/v1alpha1
kind: MaaSModelRef
metadata:
  name: gpt-4-turbo
  namespace: external-models
spec:
  modelRef:
    kind: ExternalModel
    name: gpt-4-turbo
```

2. Optional: If you need to override the auto-discovered endpoint URL, add the **endpointOverride** field:

```
apiVersion: models.opendatahub.io/v1alpha1
kind: MaaSModelRef
metadata:
  name: <model-ref-name>
  namespace: <model-namespace>
spec:
  modelRef:
    kind: <backend-kind>
    name: <backend-resource-name>
  endpointOverride: <custom-endpoint-url>
```

3. Apply the YAML file:

```
$ oc apply -f maasmodelref.yaml
```

Verification

- Verify that the model reference was created:

```
$ oc get maasmodelref <model-ref-name> -n <model-namespace>
```

- Check the model reference status:

```
$ oc get maasmodelref <model-ref-name> -n <model-namespace> -o
jsonpath={.status.phase}
```

  Expected output: **Ready**

- Verify that the model appears in the MaaS model list:

```
$ oc get maasmodelref -A
```

- Verify that the model is accessible through the MaaS gateway:

```
$ curl -H "Authorization: Bearer <api-key>" \
  https://<maas-gateway-url>/maas-api/v1/models | jq .data[].id
```

The output includes the published model ID.

NOTE

Publishing a model creates a reference for MaaS, but does not grant any user access. To make the model accessible to users, an administrator must include the model in a subscription and create a matching authorization policy.

Next steps

- Create a subscription using YAML

1.11.4. Create a subscription using YAML

You can create a Models-as-a-Service (MaaS) subscription by defining a **MaaSSubscription** custom resource in YAML and applying it using OpenShift CLI (**oc**). This approach is useful for GitOps workflows and automation.

Prerequisites

- You have cluster administrator privileges for the OpenShift cluster where OpenShift AI is installed.
- You have access to OpenShift CLI (**oc**).
- You have deployed Models-as-a-Service.
- You have published at least one model to Models-as-a-Service (MaaS).

Procedure

1. Create a YAML file defining the **MaaSSubscription** custom resource:

```
apiVersion: models.opendatahub.io/v1alpha1
kind: MaaSSubscription
metadata:
  name: <subscription-name>
  namespace: models-as-a-service
spec:
  owner:
    groups:
      - kind: Group
        name: <group-1>
      - kind: Group
        name: <group-2>
    users:
      - <user-1>
      - <user-2>
  modelRefs:
    - name: <model-name>
      namespace: <model-namespace>
      tokenRateLimits:
        - limit: <token-limit>
          window: "<time-window>"
```

```
      - limit: <token-limit>
        window: "<time-window>"
  priority: <priority-level>
```

where:

metadata.name

Specifies the Kubernetes resource name for the subscription.

metadata.namespace

Specifies the namespace where the subscription is created. This must be **models-as-a-service**.

spec.owner.groups

Lists the groups whose members can use this subscription. Each group entry requires **kind: Group** and a **name** field that references an OpenShift Group object or external OIDC group.

spec.owner.users

Lists individual users who can use this subscription. Users must be valid Kubernetes user identities.

spec.modelRefs

Defines the models available in this subscription with their token rate limits.

spec.modelRefs.name

Specifies the **MaaSModelRef** resource name.

spec.modelRefs.namespace

Specifies the namespace containing the **MaaSModelRef** resource.

spec.modelRefs.tokenRateLimits

Configures token consumption limits with time windows. At least one rate limit is required per model.

spec.modelRefs.tokenRateLimits.limit

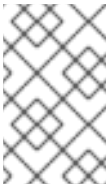
Specifies the maximum number of tokens allowed within the time window.

spec.modelRefs.tokenRateLimits.window

Specifies the time window for the rate limit. Supported units: **s** (seconds), **m** (minutes), **h** (hours). Pattern: **1-9999** followed by a unit (for example, **30s**, **5m**, **1h**, **24h**).

spec.priority

Sets the subscription priority level when a user has multiple subscriptions. Higher numbers indicate higher priority. Default: **0**.



NOTE

At least one of **groups** or **users** must be specified under **owner**. Multiple token rate limits can be configured for each model to enforce different time windows (for example, per-minute and per-day limits).

IMPORTANT

The **window** field no longer accepts **d** (days) as a unit. Use hours instead. For example, use **24h** instead of **1d**, or **168h** for a week.

Example configuration

```
apiVersion: models.opendatahub.io/v1alpha1
kind: MaaSSubscription
metadata:
  name: data-science-team
  namespace: models-as-a-service
spec:
  owner:
    groups:
      - kind: Group
        name: data-scientists
      - kind: Group
        name: ml-engineers
    users:
      - alice@example.com
  modelRefs:
    - name: llama-3-8b-instruct
      namespace: model-serving
      tokenRateLimits:
        - limit: 1000000
          window: "1h"
        - limit: 5000000
          window: "24h"
    - name: granite-7b-lab
      namespace: model-serving
      tokenRateLimits:
        - limit: 500000
          window: "1h"
  priority: 10
```

2. Apply the YAML file:

```
$ oc apply -f <subscription-file>.yaml
```

3. Create a matching authorization policy:

```
$ oc apply -f - <<EOF
apiVersion: models.opendatahub.io/v1alpha1
kind: MaaSAuthPolicy
metadata:
  name: <policy-name>
  namespace: models-as-a-service
spec:
  subjects:
    groups:
      - name: <group-1>
      - name: <group-2>
    users:
      - <user-1>
      - <user-2>
  modelRefs:
```

```
    - name: <model-name>
      namespace: <model-namespace>
EOF
```

Authorization policies are required in addition to subscriptions to grant API gateway access.

Verification

- Verify that the subscription was created:

```
$ oc get maassubscription <subscription-name> -n models-as-a-service
```

- Check the subscription status:

```
$ oc get maassubscription <subscription-name> -n models-as-a-service -o
jsonpath={.status.phase}
```

Expected output: **Ready**

Next steps

- Create an authorization policy using YAML

1.11.5. Create an authorization policy using YAML

You can create a Models-as-a-Service (MaaS) authorization policy by defining a **MaaSAuthPolicy** custom resource in YAML and applying it using OpenShift CLI (**oc**). Authorization policies grant groups and users access to model endpoints through the API gateway. Subscriptions control token quota limits separately from gateway access.

Prerequisites

- You have cluster administrator privileges for the OpenShift cluster where OpenShift AI is installed.
- You have access to OpenShift CLI (**oc**).
- You have deployed Models-as-a-Service.
- At least one MaaS subscription exists for the groups and models that you plan to authorize.

Procedure

1. Create a YAML file defining the **MaaSAuthPolicy** custom resource:

```
apiVersion: models.opendatahub.io/v1alpha1
kind: MaaSAuthPolicy
metadata:
  name: <policy-name>
  namespace: models-as-a-service
spec:
  subjects:
    groups:
      - name: <group-1>
```

```
    - name: <group-2>
  users:
    - <user-1>
    - <user-2>
modelRefs:
  - name: <model-name-1>
    namespace: <model-namespace>
  - name: <model-name-2>
    namespace: <model-namespace>
```

where:

metadata.name

Specifies the Kubernetes resource name for the authorization policy.

metadata.namespace

Specifies the namespace where the authorization policy is created. This must be **models-as-a-service**.

spec.subjects.groups

Lists the groups authorized to access model endpoints. Each group entry requires a **name** field that references a Kubernetes Group object or external OIDC group.

spec.subjects.users

Lists the individual users authorized to access model endpoints. Users must be valid Kubernetes user identities.

spec.modelRefs

Defines the model endpoints that authorized subjects can access.

spec.modelRefs.name

Specifies the **MaaSModelRef** resource name.

spec.modelRefs.namespace

Specifies the namespace containing the **MaaSModelRef** resource.

NOTE

The authorization policy uses OR logic: any matching group or user grants access to all specified models. At least one of **groups** or **users** must be specified under **subjects**.

Example configuration

```
apiVersion: models.opendatahub.io/v1alpha1
kind: MaaSAuthPolicy
metadata:
  name: ml-team-auth-policy
  namespace: models-as-a-service
spec:
  subjects:
    groups:
      - name: ml-engineers
      - name: data-scientists
    users:
      - alice@example.com
```

```
  modelRefs:
    - name: llama-3-8b-instruct
      namespace: model-serving
    - name: granite-7b-lab
      namespace: model-serving
```

2. Optional: Add metering metadata for billing attribution and cost tracking:

```
apiVersion: models.opendatahub.io/v1alpha1
kind: MaaSAuthPolicy
metadata:
  name: <policy-name>
  namespace: models-as-a-service
spec:
  subjects:
    groups:
      - name: <group-name>
  modelRefs:
    - name: <model-name>
      namespace: <model-namespace>
  meteringMetadata:
    organizationId: <organization-id>
    costCenter: <cost-center>
    labels:
      <key>: <value>
```

where:

meteringMetadata.organizationId

Specifies the organization identifier for billing purposes.

meteringMetadata.costCenter

Specifies the cost center for billing attribution.

meteringMetadata.labels

Provides additional key-value pairs for tracking and reporting.

3. Apply the YAML file:

```
$ oc apply -f <auth-policy-file>.yaml
```

IMPORTANT

Authorization policies and subscriptions are independent resources. If you modify a subscription to add or remove groups or models, you must manually update the corresponding authorization policy to keep them synchronized.

Verification

- Verify that the authorization policy was created:

```
$ oc get maasauthpolicy <policy-name> -n models-as-a-service
```

- Check the authorization policy status:

```
$ oc get maasauthpolicy <policy-name> -n models-as-a-service -o jsonpath={.status.phase}
```

  Expected output: **Active**

- Verify that authorized users can access the models:

```
$ curl -H "Authorization: Bearer <api-key>" \
  https://<maas-gateway-url>/maas-api/v1/models
```

Next steps

- Manage API keys using the Models-as-a-Service API

1.11.6. Manage API keys using the Models-as-a-Service API

In Red Hat OpenShift AI, administrators and users can create, list, and revoke API keys by using the Models-as-a-Service (MaaS) API with **curl** commands. Administrators can manage API keys for all users, while users can manage their own API keys.

Prerequisites

- You have deployed Models-as-a-Service (MaaS) on your OpenShift cluster. For more information, see Prerequisites for MaaS.
- You have access to OpenShift CLI (**oc**).
- You have published at least one model to MaaS.
- You have at least one subscription configured.
- You have an authorization policy that grants users or groups access to model endpoints.

Procedure

1. Obtain the MaaS gateway URL:

```
$ oc get gateway maas-default-gateway -n openshift-ingress -o
jsonpath='{.status.addresses[0].value}'
```

2. Obtain an authentication token:

   For OpenShift users

```
$ oc whoami -t
```

   For external OIDC users

```
$ curl -X POST "<oidc_token_endpoint>" \
  -d "client_id=<client_id>" \
  -d "client_secret=<client_secret>" \
  -d "grant_type=client_credentials"
```

3. List the available subscriptions:

```
$ curl -X GET https://<maas_gateway_url>/maas-api/v1/subscriptions \
  -H "Authorization: Bearer <auth_token>"
```

Note the subscription name you want to associate with the API key. Administrators can view all subscriptions, while regular users can view only subscriptions associated with their user or group.

4. To create an API key, send a POST request to the API keys endpoint:

```
$ curl -X POST https://<maas_gateway_url>/maas-api/v1/api-keys \
  -H "Authorization: Bearer <auth_token>" \
  -H "Content-Type: application/json" \
  -d { "name": "<key_name>", "subscription": "<subscription_name>", "expiresInDays": <days> }
```

where:

<maas_gateway_url>

Specifies your MaaS gateway URL.

<auth_token>

Specifies the authentication token obtained in step 2.

<key_name>

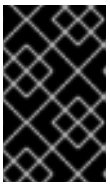
Specifies a descriptive name for the API key.

<subscription_name>

Specifies the subscription name that scopes the key's access.

<days>

Specifies the number of days until the key expires. The maximum expiration period is configured in the **Tenant** custom resource. If not set, the default maximum is 90 days.



IMPORTANT

The plain text key is returned only in this response and cannot be retrieved later. Save the key in a secrets manager or other secure storage before sending further requests.

5. To list API keys, send a GET request to the API keys endpoint:

```
$ curl -X GET https://<maas_gateway_url>/maas-api/v1/api-keys \
  -H "Authorization: Bearer <auth_token>"
```

The API returns a list of API keys with their names, subscription associations, creation dates, and expiration dates.

6. To revoke an API key, send a DELETE request to the API key endpoint:

```
$ curl -X DELETE https://<maas_gateway_url>/maas-api/v1/api-keys/<key_id> \
  -H "Authorization: Bearer <auth_token>"
```

where:

<key_id>

Specifies the ID of the API key to revoke. You can obtain the key ID from the list API keys response.

IMPORTANT

API keys are scoped to subscriptions. If a subscription is deleted, all API keys created for that subscription are invalidated.

Verification

- Verify that the API key appears in the list of keys:

```
$ curl -X GET https://<maas_gateway_url>/maas-api/v1/api-keys \
  -H "Authorization: Bearer <auth_token>"
```

- Test the API key by making a request to list available models:

```
$ curl -H "Authorization: Bearer <api_key>" \
  https://<maas_gateway_url>/maas-api/v1/models
```

Additional resources

- Models-as-a-Service API endpoints reference

1.11.7. Models-as-a-Service API endpoints reference

In Red Hat OpenShift AI, the Models-as-a-Service (MaaS) API provides programmatic access to model management, API key lifecycle operations, and subscription information. Management API endpoints are prefixed with **/maas-api/v1** and require authentication via OpenID Connect (OIDC) token or API key.

1.11.7.1. Authentication

All API requests except the health endpoint require authentication by using the **Authorization** header:

```
Authorization: Bearer <token>
```

where **<token>** is either an OpenID Connect (OIDC) JWT or a Models-as-a-Service (MaaS) API key. MaaS API keys are prefixed with **sk-oai-** and are returned by the API key creation endpoint.

1.11.7.2. GET /maas-api/health

Purpose: Cluster health check for load balancers and monitoring systems

Authentication: Not required

Example request:

```
$ curl https://<maas-gateway-url>/maas-api/health
```

Example response:

```
{
  "status": "healthy"
}
```

1.11.7.3. GET /maas-api/v1/models

Purpose: List all models accessible to the authenticated user based on their subscriptions

Authentication: Required

Headers:

X-MaaS-Subscription (optional)

When using a user token, specifies which subscription to use for filtering models. Not applicable when using API keys.

Example request:

```
$ curl -H "Authorization: Bearer <token>" \
  https://<maas-gateway-url>/maas-api/v1/models
```

Example response:

```
{
  "object": "list",
  "data": [
    {
      "id": "llama-3-8b-instruct",
      "object": "model",
      "created": 1234567890,
      "owned_by": "organization",
      "ready": true,
      "url": "https://<maas-gateway-url>/llm/llama-3-8b-instruct",
      "kind": "LLMInferenceService",
      "modelDetails": {
        "displayName": "Llama 3 8B Instruct",
        "description": "Llama 3 instruction-tuned model",
        "genaiUseCase": "chat",
        "contextWindow": 8192
      },
      "subscriptions": [
        {
          "name": "data-science-team",
          "displayName": "Data Science Team Subscription",
          "description": "Subscription for data science team"
        }
      ]
    }
  ]
}
```

1.11.7.4. POST /maas-api/v1/api-keys

Purpose: Create a new API key for programmatic access

Authentication: Required

Request body:

```
{
  "name": "<key-name>",
  "description": "<description>",
  "subscription": "<subscription-name>",
  "expiresIn": "<duration>",
  "ephemeral": <boolean>
}
```

where:

name

Specifies a human-readable name for the API key. Required for regular keys, optional for ephemeral keys.

description

Provides an optional description of the key's purpose.

subscription

Specifies the subscription name to associate with the API key. If not specified, the system selects based on user's group memberships.

expiresIn

Specifies the expiration duration as a string (for example, **30d**, **90d**, **1h**, **24h**). If not specified, keys expire after 90 days.

ephemeral

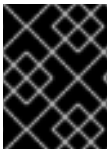
When **true**, creates a short-lived key for temporary use. Default: **false**.

Example request:

```
$ curl -X POST https://<maas-gateway-url>/maas-api/v1/api-keys \
  -H "Authorization: Bearer <token>" \
  -H "Content-Type: application/json" \
  -d { "name": "production-key", "description": "Key for production workloads", "subscription": "data-science-team", "expiresIn": "90d" }
```

Example response:

```
{
  "key": "sk-oai-abc123def456...",
  "keyPrefix": "sk-oai-abc123",
  "id": "key-abc123",
  "name": "production-key",
  "subscription": "data-science-team",
  "createdAt": "2026-05-15T12:00:00Z",
  "expiresAt": "2026-08-13T12:00:00Z",
  "ephemeral": false
}
```



IMPORTANT

The **key** field is returned only once during creation and cannot be retrieved later. Save the key in a secrets manager before continuing.

1.11.7.5. POST /maas-api/v1/api-keys/search

Purpose: Search and filter API keys with pagination and sorting support

Authentication: Required

Request body:

```
{
  "filters": {
    "username": "<username>",
    "status": ["<status-1>", "<status-2>"],
    "includeEphemeral": <boolean>
  },
  "sort": {
    "by": "<sort-field>",
    "order": "<sort-order>"
  },
  "pagination": {
    "limit": <limit>,
    "offset": <offset>
  }
}
```

where:

filters.username

Filters keys by owner username. Administrators can filter by any username; regular users can only filter their own keys.

filters.status

Filters by key status. Array of values: **active**, **revoked**, **expired**.

filters.includeEphemeral

When **true**, includes ephemeral keys in results. Default: **false**.

sort.by

Specifies the sort field. Allowed values: **created_at**, **expires_at**, **last_used_at**, **name**. Default: **created_at**.

sort.order

Specifies the sort direction. Allowed values: **asc**, **desc**. Default: **desc**.

pagination.limit

Specifies the maximum number of results to return. Default: 50. Maximum: 100.

pagination.offset

Specifies the number of results to skip for pagination. Default: 0.

Example request:

```
$ curl -X POST https://<maas-gateway-url>/maas-api/v1/api-keys/search \
  -H "Authorization: Bearer <token>" \
  -H "Content-Type: application/json" \
  -d { "filters": {"status": ["active"]}, "sort": {"by": "created_at", "order": "desc"}, "pagination": {"limit": 20, "offset": 0} }
```

Example response:

```
{
  "object": "list",
  "data": [
    {
      "id": "key-abc123",
      "name": "production-key",
      "description": "Key for production workloads",
      "username": "alice@example.com",
      "subscription": "data-science-team",
      "groups": ["data-scientists", "ml-engineers"],
      "creationDate": "2026-05-15T12:00:00Z",
      "expirationDate": "2026-08-13T12:00:00Z",
      "status": "active",
      "lastUsedAt": "2026-05-15T15:30:00Z",
      "ephemeral": false
    }
  ],
  "has_more": false
}
```

1.11.7.6. GET /maas-api/v1/api-keys/{id}

Purpose: Retrieve metadata for a specific API key

Authentication: Required

Example request:

```
$ curl -H "Authorization: Bearer <token>" \
  https://<maas-gateway-url>/maas-api/v1/api-keys/key-abc123
```

Example response:

```
{
  "id": "key-abc123",
  "name": "production-key",
  "description": "Key for production workloads",
  "username": "alice@example.com",
  "subscription": "data-science-team",
  "groups": ["data-scientists", "ml-engineers"],
  "creationDate": "2026-05-15T12:00:00Z",
  "expirationDate": "2026-08-13T12:00:00Z",
  "status": "active",
  "lastUsedAt": "2026-05-15T15:30:00Z",
  "ephemeral": false
}
```

1.11.7.7. DELETE /maas-api/v1/api-keys/{id}

Purpose: Revoke a specific API key immediately

Authentication: Required

Example request:

```
$ curl -X DELETE \
  -H "Authorization: Bearer <token>" \
  https://<maas-gateway-url>/maas-api/v1/api-keys/key-abc123
```

Example response:

```
{
  "id": "key-abc123",
  "name": "production-key",
  "description": "Key for production workloads",
  "username": "alice@example.com",
  "subscription": "data-science-team",
  "groups": ["data-scientists", "ml-engineers"],
  "creationDate": "2026-05-15T12:00:00Z",
  "expirationDate": "2026-08-13T12:00:00Z",
  "status": "revoked",
  "lastUsedAt": "2026-05-15T15:30:00Z",
  "ephemeral": false
}
```

1.11.7.8. POST /maas-api/v1/api-keys/bulk-revoke

Purpose: Revoke all active API keys for a specific user

Authentication: Required. Administrator privileges required.

Request body:

```
{
  "username": "<username>"
}
```

Example request:

```
$ curl -X POST https://<maas-gateway-url>/maas-api/v1/api-keys/bulk-revoke \
  -H "Authorization: Bearer <token>" \
  -H "Content-Type: application/json" \
  -d { "username": "alice@example.com" }
```

Example response:

```
{
  "message": "Successfully revoked 3 active API key(s) for user alice@example.com"
}
```

1.11.7.9. GET /maas-api/v1/subscriptions

Purpose: List all subscriptions accessible to the authenticated user

Authentication: Required

Example request:

```
$ curl -H "Authorization: Bearer <token>" \
  https://<maas-gateway-url>/maas-api/v1/subscriptions
```

Example response:

```
[
  {
    "subscription_id_header": "data-science-team",
    "display_name": "Data Science Team Subscription",
    "subscription_description": "Subscription for data science team members",
    "priority": 10,
    "model_refs": [
      {
        "name": "llama-3-8b-instruct",
        "namespace": "model-serving",
        "token_rate_limits": [
          {
            "limit": 1000000,
            "window": "1h"
          },
          {
            "limit": 5000000,
            "window": "24h"
          }
        ],
        "billing_rate": {
          "per_token": 0.001
        }
      }
    ],
    "organization_id": "org-123",
    "cost_center": "ml-team",
    "labels": {
      "environment": "production",
      "team": "data-science"
    }
  }
]
```

NOTE

The response is an array of subscription objects, not an object with a **data** field. Each subscription includes complete model reference details with token rate limits and billing information.

1.11.7.10. GET /maas-api/v1/model/{model-id}/subscriptions

Purpose: List all subscriptions that provide access to a specific model

Authentication: Required

Example request:

```
$ curl -H "Authorization: Bearer <token>" \
  https://<maas-gateway-url>/maas-api/v1/model/llama-3-8b-instruct/subscriptions
```

Example response:


```
[
  {
    "subscription_id_header": "data-science-team",
    "display_name": "Data Science Team Subscription",
    "subscription_description": "Subscription for data science team members",
    "priority": 10,
    "model_refs": [
      {
        "name": "llama-3-8b-instruct",
        "namespace": "model-serving",
        "token_rate_limits": [
          {
            "limit": 1000000,
            "window": "1h"
          }
        ],
        "billing_rate": {
          "per_token": 0.001
        }
      }
    ],
    "organization_id": "org-123",
    "cost_center": "ml-team",
    "labels": {
      "environment": "production"
    }
  }
]
```


1.11.7.11. Inference endpoints

Model inference endpoints use OpenAI-compatible request and response formats. Each model has a dedicated endpoint path: **https://<maas-gateway-url>/llm/{model-name}/v1**

1.11.7.12. POST /llm/{model-name}/v1/completions

Purpose: Generate text completions

Authentication: Required. API key with **sk-oai-** prefix.

Example request:

```
$ curl -X POST https://<maas-gateway-url>/llm/<model-name>/v1/completions \
```

```
  -H "Authorization: Bearer sk-oai-..." \
  -H "Content-Type: application/json" \
  -d { "model": "<model-name>", "prompt": "Explain quantum computing", "max_tokens": 100 }
```

Example response:

```
{
  "id": "cmpl-abc123",
  "object": "text_completion",
  "created": 1234567890,
  "model": "llama-3-8b-instruct",
  "choices": [
    {
      "text": "Quantum computing is a type of computing that uses quantum mechanics...",
      "index": 0,
      "finish_reason": "length"
    }
  ],
  "usage": {
    "prompt_tokens": 4,
    "completion_tokens": 100,
    "total_tokens": 104
  }
}
```

1.11.7.13. POST /llm/{model-name}/v1/chat/completions

Purpose: Generate chat-based completions

Authentication: Required. API key with **sk-oai-** prefix.

Example request:

```
$ curl -X POST https://<maas-gateway-url>/llm/<model-name>/v1/chat/completions \
  -H "Authorization: Bearer sk-oai-..." \
  -H "Content-Type: application/json" \
  -d { "model": "<model-name>", "messages": [ {"role": "user", "content": "What is AI?"} ],
"max_tokens": 100 }
```

Example response:

```
{
  "id": "chatcmpl-abc123",
  "object": "chat.completion",
  "created": 1234567890,
  "model": "llama-3-8b-instruct",
  "choices": [
    {
      "index": 0,
      "message": {
        "role": "assistant",
        "content": "Artificial Intelligence (AI) is the simulation of human intelligence..."
      },
      "finish_reason": "length"
    }
```

```
  ],
  "usage": {
    "prompt_tokens": 12,
    "completion_tokens": 100,
    "total_tokens": 112
  }
}
```

1.11.7.14. POST /llm/{model-name}/v1/embeddings

Purpose: Generate text embeddings

Authentication: Required. API key with **sk-oai-** prefix.

Example request:

```
$ curl -X POST https://<maas-gateway-url>/llm/<model-name>/v1/embeddings \
  -H "Authorization: Bearer sk-oai-..." \
  -H "Content-Type: application/json" \
  -d { "model": "<model-name>", "input": "The quick brown fox" }
```

Example response:

```
{
  "object": "list",
  "data": [
    {
      "object": "embedding",
      "embedding": [0.0023, -0.0142, 0.0089, ...],
      "index": 0
    }
  ],
  "model": "llama-3-8b-instruct",
  "usage": {
    "prompt_tokens": 4,
    "total_tokens": 4
  }
}
```

NOTE

Response formats follow the OpenAI API specification. For complete API reference documentation, see OpenAI API Reference.

1.11.7.15. Error responses

All API endpoints return standard HTTP status codes and JSON error responses:

```
{
  "error": {
    "message": "Invalid API key",
    "type": "authentication_error",
```

```
    "code": "invalid_api_key"
  }
}
```

Common HTTP status codes:

Status	Description
200 OK	Request successful
201 Created	Resource created successfully
400 Bad Request	Invalid request parameters
401 Unauthorized	Missing or invalid authentication
403 Forbidden	Insufficient permissions
404 Not Found	Resource not found
429 Too Many Requests	Rate limit exceeded
500 Internal Server Error	Server error

1.12. MONITOR MODELS-AS-A-SERVICE USAGE WITH OBSERVABILITY

You can use the MaaS observability dashboard to monitor subscription-level usage metrics for cost attribution and showback reporting to finance teams.

IMPORTANT

The MaaS observability dashboard is a Technology Preview feature in Red Hat OpenShift AI. Technology Preview features are not supported with Red Hat production service level agreements (SLAs) and might not be functionally complete. Red Hat does not recommend using them in production.

For more information about the support scope of Red Hat Technology Preview features, see Technology Preview Features Support Scope .

1.12.1. Models-as-a-Service observability overview

In Red Hat OpenShift AI, you can view Models-as-a-Service (MaaS) subscription-level usage metrics in the observability dashboard, including token consumption, request counts, and rate limit violations.

The dashboard is embedded in the OpenShift AI console using Perses and queries metrics from Thanos Querier. Access to the dashboard is restricted to cluster administrators.

NOTE

The observability dashboard is designed for showback reporting, not as a billing-grade metering system. For production chargeback workflows that require precise billing data, access the **Limitador** metrics endpoint directly rather than using Prometheus or the dashboard CSV export.

The dashboard provides the following capabilities:

Overview metrics

View high-level statistics including total tokens consumed, total requests, total errors, success rate percentage, and active users.

Filtering

Filter metrics by user, subscription, and model to analyze specific usage patterns.

Time range selection

View metrics for configurable time periods ranging from the last 5 minutes to the last 14 days, or specify a custom date range.

Token consumption details

View a detailed table showing token consumption by user, subscription, and model, including request counts and rate limit violations.

Data export

Export usage data in CSV format for cost attribution and showback reporting to finance teams.

1.12.1.1. Dashboard metrics

The observability dashboard displays the following overview metrics:

Table 1.3. Overview metrics

Metric	Description
Total Tokens	The total number of tokens consumed across all requests during the selected time period. This includes both input tokens (from user prompts) and output tokens (from model responses).
Total Requests	The total number of API requests made to MaaS models during the selected time period. Each API call to a model endpoint counts as one request.
Total Errors	The total number of failed requests during the selected time period. This includes requests that failed due to model errors, timeout errors, or other server-side issues.
Success Rate	The percentage of successful requests out of all requests made during the selected time period. Calculated as: **(Total Requests - Total Errors) / Total Requests × 100**.
Active Users	The number of unique users who made at least one request during the selected time period. Users are identified by their username from API key ownership or OIDC authentication.

The Token Consumption by User table displays detailed, per-user usage data with the following columns:

Table 1.4. Token consumption table columns

Column	Description
User	The username of the user who made the requests. For API key-based requests, this is the user who created the API key. For OIDC-authenticated requests, this is the user's OIDC identity.
Subscription	The subscription used for the requests. If a user belongs to multiple subscriptions, separate rows appear for each subscription.
Model	The model accessed by the user's requests. The format is **<endpoint-name>/<model-id>** for MaaS models.
Tokens	The total number of tokens consumed by this user for this subscription and model combination. Click the column header to sort the table by token consumption.
Requests	The number of API requests made by this user for this subscription and model combination.
Rate Limited	The number of requests that were rejected due to rate limiting (HTTP 429 responses). Rate-limited requests count toward the user's request total without consuming tokens.

1.12.1.2. Underlying Prometheus metrics

The observability dashboard queries the following Prometheus metrics collected from **Kuadrant** and MaaS components:

Table 1.5. MaaS Prometheus metrics

Metric	Description
authorized_hits	Total number of tokens consumed. Labeled by subscription, model, and limitador_namespace. This metric is collected from **Limitador** and represents the actual token usage for successful requests.
authorized_calls	Total number of API requests made to models. Labeled by subscription and limitador_namespace. This metric counts all successful requests that passed rate limiting and authentication.
limited_calls	Total number of requests rejected due to rate limiting (HTTP 429 responses). Labeled by limitador_namespace. This metric indicates when users exceed their subscription token limits.

Metric	Description
istio_request_duration_milliseconds_bucket	Request latency at the API gateway. Tagged with subscription dimension for performance analysis. This metric helps identify performance issues by subscription.
auth_server_authconfig_duration_seconds	Time spent in **Authorino** authentication and authorization. Labeled by authorization policy. This metric helps troubleshoot authentication performance.

NOTE

The **user** label is disabled by default in MaaS metrics (**captureUser: false**). To enable per-user metrics collection, configure the **captureUser** setting in your MaaS authorization policy. The **model** label appears only on the **authorized_hits** metric due to Kuadrant wasm-shim limitations.

These metrics are scraped by Prometheus from the Limitador, Authorino, and gateway components. The observability dashboard aggregates and visualizes these metrics to provide usage insights for cost attribution and capacity planning.

1.12.2. Enable Kuadrant observability for Models-as-a-Service

In Red Hat OpenShift AI, you can enable observability in the **Kuadrant** custom resource to collect rate-limiting metrics for Models-as-a-Service usage tracking and monitoring.

Kuadrant uses **Limitador**, a rate-limiting service, to enforce the token limits defined in Models-as-a-Service (MaaS) subscriptions. When observability is enabled, Kuadrant creates a **PodMonitor** that configures Prometheus to scrape metrics from **Limitador**. These metrics track token consumption, request counts, and rate-limit violations, which are displayed in the MaaS observability dashboard for cost attribution and usage monitoring.

Prerequisites

- You have cluster administrator privileges for the OpenShift cluster where OpenShift AI is installed.
- You have installed Red Hat OpenShift AI.
- You have configured the observability stack for OpenShift AI. For more information, see Managing observability.

Procedure

1. Enable Kuadrant observability using the OpenShift console:
   a. In the OpenShift console, navigate to **Administration → CustomResourceDefinitions**.
   b. Search for **Kuadrant** and click the resource name.
   c. Click the **Instances** tab.

d. Click **kuadrant**.

e. Click the **YAML** tab.

f. In the YAML editor, locate the **spec** section and add or update the observability configuration:

```
spec:
  observability:
    enable: true
```

The patch uses the following fields:

enable: true

Enables the **Kuadrant** observability stack, which creates a **PodMonitor** resource that configures Prometheus to scrape rate-limiting metrics from **Limitador**. These metrics are used by the MaaS observability dashboard to track token consumption, request counts, and rate-limit violations.

g. Click **Save**.
Alternatively, you can configure the resource using the command line:

```
$ oc patch kuadrant kuadrant -n kuadrant-system \
  --type merge \
  -p '{"spec":{"observability":{"enable":true}}}'
```

Verification

- Verify that the **Limitador PodMonitor** was created:

```
$ oc get podmonitor kuadrant-limitador-monitor -n kuadrant-system
```

Expected output shows the **PodMonitor** resource:

```
NAME                        AGE
kuadrant-limitador-monitor   2m
```

- Verify that the **Kuadrant** custom resource shows observability as enabled:

```
$ oc get kuadrant kuadrant -n kuadrant-system -o jsonpath='{.spec.observability.enable}'
```

Expected output:

```
true
```

- Verify that Prometheus is scraping metrics from **Limitador**:

a. In the OpenShift console, navigate to **Observe → Metrics**.

b. Run the following query to verify rate-limiting metrics are available:

```
limited_calls
```

You should see metrics showing rate-limit violations by user and subscription. If no data appears, this is expected if no models have been accessed yet. The metric appears once users begin making requests to MaaS models.

Next steps

- Enable telemetry for Models-as-a-Service

1.12.3. Enable telemetry for Models-as-a-Service

In Red Hat OpenShift AI, you can enable telemetry in the **Tenant** custom resource to collect usage metrics from Models-as-a-Service (MaaS) inference requests.

Telemetry configures the MaaS gateway to generate Prometheus metrics about model usage, including token consumption, request counts, and model-specific usage patterns. These metrics are displayed in the MaaS observability dashboard.

Prerequisites

- You have cluster administrator privileges for the OpenShift cluster where OpenShift AI is installed.
- You have installed Red Hat OpenShift AI.
- You have deployed Models-as-a-Service.

Procedure

1. Enable telemetry using the OpenShift console:
   a. In the OpenShift console, navigate to **Administration → CustomResourceDefinitions**.
   b. Search for **Tenant** and click the resource name.
   c. Click the **Instances** tab.
   d. Click **default-tenant**.
   e. Click the **YAML** tab.
   f. In the YAML editor, locate the **spec** section and add or update the **telemetry** configuration:

```
spec:
  telemetry:
    enabled: true
    metrics:
      captureOrganization: true
      captureUser: false
      captureGroup: false
      captureModelUsage: true
```

The patch uses the following fields:

enabled: true

Activates TelemetryPolicy and Istio Telemetry to collect MaaS usage metrics.

captureOrganization

Includes organization identifiers in metrics. Default is **true**.

captureUser

Includes user labels in metrics. Default is **false** due to privacy and cardinality considerations. Enabling this option with a large number of users can significantly increase Prometheus database size.

captureGroup

Includes group labels in metrics. Default is **false** to reduce metric cardinality.

captureModelUsage

Tracks model-specific usage patterns. Default is **true**.

g. Click **Save**.
Alternatively, you can configure the resource using the command line:

```
$ oc patch tenants.maas.opendatahub.io default-tenant -n models-as-a-service \
  --type merge \
  -p '{
    "spec": {
      "telemetry": {
        "enabled": true,
        "metrics": {
          "captureOrganization": true,
          "captureUser": false,
          "captureGroup": false,
          "captureModelUsage": true
        }
      }
    }
  }'
```

Verification

1. In the OpenShift console, navigate to **Observe → Metrics**.
2. In the query field, enter the following metric name:

```
authorized_calls
```

3. Click **Run queries**.

If telemetry is enabled, the query returns MaaS usage metrics with labels such as **subscription**, **limitador_namespace**, and optionally **model** depending on your telemetry configuration.

1.12.4. View the Models-as-a-Service observability dashboard

In Red Hat OpenShift AI, you can use the Models-as-a-Service (MaaS) observability dashboard to monitor token consumption, request counts, and rate-limit violations across subscriptions and users.

IMPORTANT

The MaaS observability dashboard is intended for internal usage tracking and showback reporting. The metrics are not suitable for billing-grade metering or external invoicing.

Prerequisites

- You have cluster administrator privileges for the OpenShift cluster where OpenShift AI is installed.
- You have installed Red Hat OpenShift AI.
- You have configured observability for Models-as-a-Service. For more information, see Prerequisites for Models-as-a-Service.
- You have enabled Kuadrant observability. For more information, see Enable Kuadrant observability for Models-as-a-Service.
- You have enabled telemetry for Models-as-a-Service. For more information, see Enable telemetry for Models-as-a-Service.

Procedure

1. In the OpenShift AI dashboard, click **Observe & monitor → Dashboard** in the left navigation menu.
   The Observability dashboard page displays three tabs: **Cluster**, **Models**, and **Usage**.
2. Click the **Usage** tab to view Models-as-a-Service usage metrics.
3. Optional: To change the time range, select a value from the **Time period** dropdown. Options range from the last 5 minutes to the last 14 days. You can also specify a custom date range.
4. Optional: Filter the metrics by user, subscription, or model by selecting values from the **User**, **Subscription**, or **Model** dropdowns. Select **All** in any dropdown to view metrics for all items in that category.
5. Review the **Overview** section, which displays summary metrics including **Total Tokens**, **Total Requests**, **Total Errors**, **Success Rate**, and **Active Users**.
6. Review the **Token Consumption by User** table, which shows detailed per-user usage data.
7. Optional: Click column headers in the table to sort by that column.
8. Optional: Use the pagination controls at the bottom of the table to navigate through multiple pages of results or adjust the number of rows displayed per page.

Verification

- The **Overview** section shows non-zero values for users with recent activity.
- The **Token Consumption by User** table lists users with token consumption in the selected time period.
- Changing the time period updates the metrics to reflect the new range.

Additional resources

- Models-as-a-Service observability overview

Next steps

- Export usage data
- Managing subscriptions for Models-as-a-Service

1.12.5. Export usage data for cost attribution

In Red Hat OpenShift AI, you can export Models-as-a-Service usage data in CSV format for cost attribution and showback reporting to finance teams.

Prerequisites

- You have cluster administrator privileges for the OpenShift cluster where OpenShift AI is installed.
- The Cluster Observability Operator is installed and configured on your cluster.
- You have access to the OpenShift AI dashboard with administrator privileges.

Procedure

1. In the OpenShift AI dashboard, click **Observe & monitor → Dashboard**.
2. Click the **Usage** tab.
3. Optional: Configure filters to export specific usage data:
   a. Select a time period from the **Time period** dropdown to export metrics for a specific timeframe.
   b. Select filters for **User**, **Subscription**, or **Model** to export metrics for specific resources.
4. Hover over the Token Consumption by User table.
5. Click **Export as CSV** to download the usage data.
   The system generates a CSV file containing the filtered usage data.
6. Save the CSV file to your local system.

NOTE

The exported CSV file contains subscription-level usage data for the selected time period and filters. This data is suitable for showback reporting but might not be billing-grade accurate. For production chargeback workflows, configure external metering and billing tools to consume this data.

Verification

- The CSV file downloads to your local system and contains usage data matching your selected filters and time period.

Next steps

- Provide the exported usage data to your finance team for cost attribution and showback reporting.

Additional resources

- Subscriptions
- Models-as-a-Service administration troubleshooting

1.13. CONFIGURE EXTERNAL OIDC AUTHENTICATION FOR MODELS-AS-A-SERVICE

IMPORTANT

External OIDC authentication for Models-as-a-Service is currently available in Red Hat OpenShift AI as a Technology Preview feature. Technology Preview features are not supported with Red Hat production service level agreements (SLAs) and might not be functionally complete. Red Hat does not recommend using them in production. These features provide early access to upcoming product features, enabling customers to test functionality and provide feedback during the development process.

For more information about the support scope of Red Hat Technology Preview features, see Technology Preview Features Support Scope .

You can configure Models-as-a-Service (MaaS) to authenticate users with an external OpenID Connect (OIDC) identity provider, enabling enterprise-wide access without requiring OpenShift accounts for every user. This allows organizations to integrate MaaS with existing identity providers such as Keycloak, and map external user groups to MaaS subscriptions for access control and quota enforcement.

1.13.1. About external OIDC authentication for Models-as-a-Service

In Red Hat OpenShift AI, you can authenticate Models-as-a-Service users through an external OpenID Connect (OIDC) identity provider, allowing them to use their existing corporate credentials without requiring OpenShift user accounts.

1.13.1.1. Authentication flow

Models-as-a-Service (MaaS) uses a two-tier authentication approach with external OIDC providers as follows:

1. **MaaS platform access**: Users retrieve a JWT from the external OIDC provider to access platform APIs. The **Gateway** API validates the OIDC token.
2. **Model access**: Users create API keys through the MaaS API by using their OIDC token. These API keys are used for programmatic access to models through the MaaS API gateway.

NOTE

When using external OIDC authentication, users create API keys through the MaaS API by using **curl** or other HTTP clients. The OpenShift AI dashboard does not support API key creation for external OIDC users.

This approach provides industry-standard OIDC authentication for user login while maintaining centralized API key management for model access.

1.13.1.2. Group-based access control

MaaS validates external identity provider groups directly from OIDC tokens to determine user access. The OIDC token must include group claims for authorization to work. The validation process follows these steps:

1. **OIDC provider** defines user groups.
2. **OIDC token** includes group claims when a user authenticates. For example, a token might include **groups: ["data-scientists", "ml-engineers"]**. The **groups** claim is required for MaaS authorization.
3. **MaaS subscriptions** define which groups have access to specific models. For example, a subscription might grant access to the **data-scientists** group.
4. **Authorization policies** validate the group claims in the user's OIDC token against the groups defined in subscriptions. If the token includes **data-scientists** and the subscription grants access to that group, the user is authorized.

Group names in MaaS subscriptions and authorization policies must match the group names in the OIDC token claims exactly. MaaS validates groups directly from the token without requiring OpenShift group creation or synchronization.

1.13.1.3. API key lifecycle

The API key creation process for OIDC-authenticated users follows these steps:

1. Users retrieve a JWT from the external OIDC provider.
2. Users call the MaaS API with their OIDC token to create an API key.
3. The **Gateway** validates the OIDC token.
4. MaaS generates an API key with an expiration period specified in the API request, up to the maximum limit configured in the **Tenant** custom resource.
5. Users use this API key for model access.

IMPORTANT

API keys capture the user's group memberships at the time of creation. If a user is removed from a group in the external OIDC provider, their existing API keys retain the original group associations and continue to work until revoked or expired. To immediately revoke access, administrators must manually revoke the user's API keys.

1.13.1.4. Use cases

Enterprise deployment: Organizations with existing identity providers such as Keycloak can integrate MaaS without creating OpenShift accounts for every user, reducing the overhead of managing a large user base.

Service provider deployment: Service providers offering MaaS to external customers can authenticate users through a centralized OIDC provider while maintaining subscription-based isolation and quota enforcement.

Regulated industries: Organizations with compliance requirements for centralized authentication and audit logging can use external OIDC integration while maintaining MaaS governance features.

1.13.2. Configure Models-as-a-Service for external OIDC users

In Red Hat OpenShift AI, you can configure Models-as-a-Service (MaaS) to authenticate users through an external OpenID Connect (OIDC) identity provider to enforce group-based access control without requiring OpenShift accounts for every user.

Prerequisites

- You have cluster administrator privileges for the OpenShift cluster where OpenShift AI is installed.
- You have installed Red Hat OpenShift AI.
- You have deployed Models-as-a-Service.
- You have an external OIDC provider.
- You have registered a client application in your OIDC provider and obtained the issuer URL and client ID.
- You have created user groups in your external OIDC provider.
- Your external OIDC provider is configured to include group claims in ID tokens.

Procedure

1. Edit the **Tenant** custom resource to add the **externalOIDC** configuration, using one of the following methods:

   Using the OpenShift console

   a. In the OpenShift console, navigate to **Administration → CustomResourceDefinitions**.

   b. Search for **Tenant** and click the resource name.

   c. Click the **Instances** tab.

   d. Click **default-tenant**.

   e. Click the **YAML** tab.

   f. In the YAML editor, locate the **spec** section and add or update the **externalOIDC** configuration:

```
spec:
  externalOIDC:
    issuerUrl: <oidc-provider-issuer-url>
    clientId: <oidc-client-id>
```

The configuration uses the following fields:

<oidc-provider-issuer-url>

Specifies the endpoint URL for your external identity provider.

<oidc-client-id>

Specifies the client ID for your MaaS application registered with the OIDC provider.

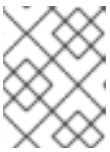
a. Click **Save**.

Using the OpenShift CLI

Run the following command:

```
$ oc patch tenants.maas.opendatahub.io default-tenant -n models-as-a-service \
  --type merge \
  -p { "spec": { "externalOIDC": { "issuerUrl": "<oidc-provider-issuer-url>", "clientId": "<oidc-client-id>" } } }
```

2. Create MaaS subscriptions that include the groups from your OIDC provider.



NOTE

Group names in subscriptions must match the group names in the OIDC token claims exactly. MaaS validates group memberships directly from the OIDC token.

When configuring subscriptions, enter group names exactly as they appear in your OIDC provider's group claims. For example, if your OIDC token includes **groups: ["data-scientists"]**, enter **data-scientists** in the subscription.

For information about creating subscriptions, see Managing subscriptions for Models-as-a-Service.

Verification

To verify external OIDC authentication, obtain a token from your OIDC provider and use it to access the MaaS API:

1. Obtain an OIDC token from your identity provider using the OAuth 2.0 client credentials grant:

```
$ curl -X POST "<oidc-token-endpoint>" \
  -d "client_id=<client-id>" \
  -d "client_secret=<client-secret>" \
  -d "grant_type=client_credentials"
```

The configuration uses the following fields:

<oidc-token-endpoint>

Specifies the token endpoint URL from your OIDC provider.

<client-id>

Specifies your OIDC client ID.

<client-secret>

Specifies your OIDC client secret.

2. Use the OIDC token to list available models:

```
$ curl -H "Authorization: Bearer <oidc-token>" \
  https://<maas-gateway-url>/maas-api/v1/models
```

The configuration uses the following fields:

<oidc-token>

Specifies the JWT obtained from your OIDC provider.

<maas-gateway-url>

Specifies your MaaS gateway URL.
If authentication is successful, the API returns a list of models available to the groups in your token.

IMPORTANT

API keys capture group memberships at creation time. If a user is removed from a group in the external OIDC provider, their existing API keys continue to work until revoked or expired. Administrators must manually revoke API keys to immediately revoke access.

Next steps

- Create an API key

Additional resources

- Manage API keys using the MaaS API
- Create a subscription using YAML

1.14. CONFIGURE EXTERNAL MODELS FOR MODELS-AS-A-SERVICE

IMPORTANT

External models for Models-as-a-Service is currently available in Red Hat OpenShift AI as a Technology Preview feature. Technology Preview features are not supported with Red Hat production service level agreements (SLAs) and might not be functionally complete. Red Hat does not recommend using them in production. These features provide early access to upcoming product features, enabling customers to test functionality and provide feedback during the development process.

For more information about the support scope of Red Hat Technology Preview features, see Technology Preview Features Support Scope .

You can configure Models-as-a-Service (MaaS) to route inference requests to models hosted by external cloud providers such as OpenAI, Anthropic, AWS Bedrock, Azure OpenAI, or Google Vertex AI. This enables unified governance and authentication for both locally deployed models and external model endpoints through the same MaaS gateway.

1.14.1. About external models for Models-as-a-Service

In Red Hat OpenShift AI, you can use Models-as-a-Service (MaaS) external models to route inference requests to large language models hosted outside the cluster, such as OpenAI, Anthropic, AWS Bedrock, Azure OpenAI, and Google Vertex AI, through the same MaaS gateway you use for locally deployed models.

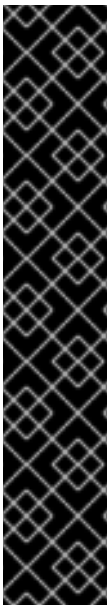
External models appear in the MaaS dashboard alongside locally deployed models. Users access external models the same way as locally deployed models: administrators include the model in a subscription, users create a MaaS API key, and users make requests using the OpenAI-compatible API format. External models differ from locally deployed models in two ways: external models use a two-tier authentication pattern, and token limits at the external provider level are shared across all users.

1.14.1.1. Two-tier authentication

External model access uses a two-tier authentication pattern:

1. **User authentication**: Users authenticate to MaaS by using their MaaS API key. MaaS validates the user subscription and confirms access to the requested external model.
2. **Provider authentication**: MaaS automatically injects the provider API key from the Kubernetes secret when forwarding requests to the external provider.

As a result, users need only their MaaS API key to access external models. The provider API key is managed by administrators and shared across all users of that external model.



IMPORTANT

Token limits apply at two levels:

- **MaaS subscription level**: Token limits configured in MaaS subscriptions apply per-user within MaaS. These limits control individual user consumption.
- **External provider level**: Token limits imposed by the external provider on the API key apply to the aggregate consumption of all users of that external model. Because all users share the same provider API key, the provider-level limit is shared across the entire group of users.

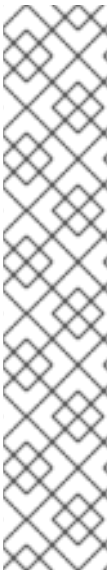
Administrators must ensure that the token limit on the provider API key can handle the combined consumption of all users who access the external model. If the provider-level limit is exceeded, no users can access the model until the provider resets the limit, which is typically hourly, daily, or monthly depending on the provider. Individual MaaS subscription limits do not affect this provider-level restriction.

Additional resources

- Models-as-a-Service subscriptions
- Authorization policies
- Create a subscription for Models-as-a-Service
- Create an API key
- Configure routing to external model providers

1.14.2. Configure routing to external model providers

In Red Hat OpenShift AI, you can configure Models-as-a-Service (MaaS) to route inference requests to large language models (LLMs) hosted by external cloud providers such as OpenAI, Anthropic, AWS Bedrock, Azure OpenAI, or Google Vertex AI, enabling unified governance for both locally deployed models and external models. Users access external models through MaaS using their MaaS API keys. The provider API key is used by MaaS to authenticate requests to the external provider.



NOTE

Token limits apply at two levels:

- **MaaS subscription level**: Token limits you configure in MaaS subscriptions control per-user consumption within MaaS.
- **External provider level**: Token limits on the provider API key, configured by the external provider, apply to the aggregate consumption of all users accessing the external model. All users share the same provider API key, so the provider-level limit is shared across all users.

Make sure that the token limit on the provider API key can handle the combined consumption of all users who access the external model. If the provider-level limit is exceeded, no users can access the model until the provider resets the limit, which is typically hourly, daily, or monthly depending on the provider.

Prerequisites

- You have cluster administrator privileges for the OpenShift cluster where OpenShift AI is installed.
- You have installed Red Hat OpenShift AI.
- You have deployed Models-as-a-Service.
- You have created at least one MaaS subscription to which you will add the external model.
- You have identified the external model provider, API endpoint hostname, and target model ID. For example, provider **openai**, endpoint **api.openai.com**, and target model **gpt-4o**.
- You have an API key for the external model provider.
- You have created a model namespace, such as **llm**.

Procedure

1. Create a Kubernetes secret with the external provider API key:

```
$ oc create secret generic <provider-api-key-secret> \
  --from-literal=api-key=<provider-api-key> \
  --labels=inference.networking.k8s.io/bbr-managed=true \
  -n <model-namespace>
```

The command uses the following placeholders:

<provider-api-key-secret>

Specifies the name of the secret containing the provider API key.

<provider-api-key>

Specifies the API key for the external provider.

<model-namespace>

Specifies the name of the model namespace you created, such as **llm**.
This secret stores the provider API key that MaaS uses to authenticate requests to the external model.

The resulting secret has the following structure:

```
apiVersion: v1
kind: Secret
metadata:
  name: <provider-api-key-secret>
  namespace: <model-namespace>
  labels:
    inference.networking.k8s.io/bbr-managed: "true"
type: Opaque
data:
  api-key: <base64-encoded-api-key>
```

2. Create an **ExternalModel** custom resource, using one of the following methods:

Using the OpenShift console

a. In the OpenShift console, navigate to **Administration → CustomResourceDefinitions**.

b. Search for **ExternalModel** and click the resource name.

c. Click the **Instances** tab.

d. From the **Project** dropdown, select your model namespace, such as **llm**.

e. Click **Create ExternalModel**.

f. In the YAML editor, replace the default content with the following configuration:

```
apiVersion: maas.opendatahub.io/v1alpha1
kind: ExternalModel
metadata:
  name: <external-model-name>
  namespace: <model-namespace>
spec:
  provider: <provider-type>
  endpoint: <external-provider-hostname>
  targetModel: <target-model-id>
  credentialRef:
    name: <provider-api-key-secret>
```

The YAML file uses the following placeholders:

<external-model-name>

Specifies the name for the external model resource.

<provider-type>

Specifies the provider type. Allowed values: **openai**, **anthropic**, **azure-openai**, **vertex**, **bedrock-openai**.

<external-provider-hostname>

Specifies the fully qualified domain name (FQDN) of the external provider without scheme or path, such as **api.openai.com**.

<target-model-id>

Specifies the upstream model identifier at the provider, such as **gpt-4o**.

<provider-api-key-secret>

Specifies the name of the secret created in the previous step.

<model-namespace>

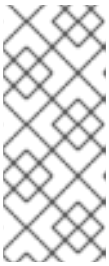
Specifies the name of the model namespace you created, such as **llm**.

a. Click **Create**.

Using the OpenShift CLI

Run the following command:

```
$ cat <<EOF | oc apply -f -
apiVersion: maas.opendatahub.io/v1alpha1
kind: ExternalModel
metadata:
  name: <external-model-name>
  namespace: <model-namespace>
spec:
  provider: <provider-type>
  endpoint: <external-provider-hostname>
  targetModel: <target-model-id>
  credentialRef:
    name: <provider-api-key-secret>
EOF
```



NOTE

When the **ExternalModel** resource is created, the MaaS controller automatically creates the required networking resources: **Service**, **HTTPRoute**, **ServiceEntry**, and **DestinationRule**. These resources enable routing from the MaaS gateway to the external provider. If external model routing fails, verify that these resources were created successfully in your model namespace.

3. Create a **MaaSModelRef** custom resource in the same namespace as the **ExternalModel** to publish the external model to MaaS:

```
$ cat <<EOF | oc apply -f -
apiVersion: models.opendatahub.io/v1alpha1
kind: MaaSModelRef
metadata:
  name: <external-model-name>
  namespace: <model-namespace>
```

```
spec:
  modelRef:
    kind: ExternalModel
    name: <external-model-name>
EOF
```

where:

<external-model-name>

Specifies the name of the **ExternalModel** resource you created in the previous step.

<model-namespace>

Specifies the namespace where you created the **ExternalModel** resource.

4. Add the external model to a MaaS subscription and authorization policy.
When creating or updating subscriptions, the external model appears in the model list alongside locally deployed models. Select the external model and configure token limits the same way as for internal models. You must also create a matching authorization policy to grant user groups access to the external model.

   For more information, see Managing subscriptions for Models-as-a-Service and Create an authorization policy.

Verification

1. Verify that the external model was added to the subscription:
   a. In the OpenShift AI dashboard, navigate to **Settings → Subscriptions**.
   b. Click the subscription name.
   c. In the **Models** section, verify that the external model appears in the list.
2. Log in to the OpenShift AI dashboard as a user who belongs to a group included in the subscription.
3. Click **Gen AI studio → AI asset endpoints**.
4. Verify that the external model appears in the model list.
5. Create a MaaS API key and make a test inference request to the external model:

```
$ curl -X POST https://<maas-gateway-url>/maas-api/v1/chat/completions \
  -H "Authorization: Bearer <maas-api-key>" \
  -H "Content-Type: application/json" \
  -d { "model": "<external-model-name>", "messages": [{"role": "user", "content": "Hello"}] }
```

The command uses the following placeholders:

<maas-api-key>

Specifies your MaaS API key created in the dashboard.

<maas-gateway-url>

Specifies your MaaS gateway URL.

<external-model-name>

Specifies the name of the external model resource you created.
A successful response confirms that MaaS routed the request to the external provider and returns the model completion.

Additional resources

- Managing subscriptions for Models-as-a-Service
- Accessing models with Models-as-a-Service

1.15. MODELS-AS-A-SERVICE ADMINISTRATION TROUBLESHOOTING

As a OpenShift AI administrator, you can diagnose and resolve common administrative issues with Models-as-a-Service (MaaS) deployment, configuration, and management.

1.15.1. Component enablement issues

If the **maas-api** pod fails to start or shows errors after enabling the MaaS component:

- Check the pod logs for error messages:

```
$ oc logs -n redhat-ods-applications -l app.kubernetes.io/name=maas-api
```

- Verify that all prerequisites are met, especially:
  - **Kuadrant** is running in the **kuadrant-system** namespace
  - The **maas-default-gateway Gateway** exists in the **openshift-ingress** namespace
  - KServe component is set to **Managed** in the DataScienceCluster
    If **Kuadrant** is not in a ready state:
    a. Check the **Kuadrant** Operator status:

```
$ oc get kuadrant -n kuadrant-system
```

    b. If the **Kuadrant** resource shows a non-ready status, restart the **Kuadrant** Operator:
       i. In the OpenShift console, navigate to **Operators → Installed Operators**.
       ii. Select the **kuadrant-system** namespace.
       iii. Click **Red Hat Connectivity Link**.
       iv. From the **Actions** menu, click **Restart**.
    c. Wait for the Operator to restart and verify that Kuadrant becomes ready:

```
$ oc wait Kuadrant -n kuadrant-system kuadrant --for=condition=Ready --
timeout=5m
```

- Check for events related to the MaaS deployment:

```
$ oc get events -n redhat-ods-applications --sort-by='.lastTimestamp' | grep maas
```

- Verify that the required RBAC resources were created:

```
$ oc get clusterrole | grep maas
$ oc get clusterrolebinding | grep maas
```

1.15.2. Dashboard visibility issues

If MaaS features do not appear in the dashboard:

- Verify that the MaaS API component is running:

```
$ oc get pods -n redhat-ods-applications -l app.kubernetes.io/name=maas-api
```

- Check that the OdhDashboardConfig was updated correctly:

```
$ oc get odhdashboardconfig odh-dashboard-config -n redhat-ods-applications -o yaml | grep -A 2 "dashboardConfig:"
```

  Verify that **modelAsService: true** for admin features (Subscriptions, Authorization Policies) and **genAiStudio: true** for user-facing features (Models tab in AI asset endpoints).

- Clear your browser cache and hard refresh the dashboard (Ctrl+Shift+R or Cmd+Shift+R).
- Check the dashboard pod logs for errors:

```
$ oc logs -n redhat-ods-applications $(oc get pods -n redhat-ods-applications -o name | grep dashboard | head -1) --tail=50
```

- Verify that you have the required permissions to view MaaS features. Admin features require administrator access.

1.15.3. Model visibility issues

If a model is missing from the available models for MaaS:

- Verify that you selected **Publish as MaaS** in the Advanced settings during deployment.
- Check that the **MaaSModelRef** was created:

```
$ oc get maasmodelref -n <your-project-namespace>
```

- Check that the MaaS API is running:

```
$ oc get pods -n redhat-ods-applications -l app.kubernetes.io/name=maas-api
```

- Verify that the model deployment is in a Ready state:

```
$ oc get llminferenceservice -n <your-project-namespace>
```

1.15.4. User access errors: 403 Forbidden

If users receive 403 Forbidden errors when accessing models through MaaS:

- Verify that the user has both a subscription and an authorization policy:
  - A subscription grants quota for specific models with token limits.
  - An authorization policy is required to authorize groups to access model endpoints through the API gateway.
- Check that a subscription exists for the user's groups:

```
$ oc get maassubscriptions -n redhat-ods-applications
```

- Verify that the model is included in the subscription:

```
$ oc get maassubscription <subscription-name> -n redhat-ods-applications -o yaml
```

- Check that an authorization policy exists:

```
$ oc get maasauthpolicies -n redhat-ods-applications
```

- Verify that the authorization policy includes the user's groups:

```
$ oc get maasauthpolicy <policy-name> -n redhat-ods-applications -o yaml
```

1.15.5. Subscription access control issues

If users receive unexpected access denials:

- Verify that the subscription status is **Active**:

```
$ oc get maassubscription <subscription-name> -n redhat-ods-applications -o
jsonpath='{.status.phase}'
```

- Check the subscription conditions for errors:

```
$ oc get maassubscription <subscription-name> -n redhat-ods-applications -o
jsonpath='{.status.conditions}'
```

- Ensure the model deployment is ready:

```
$ oc get llminferenceservice -n <model-namespace>
```

- Check the **Gateway** logs for authorization errors:

```
$ oc logs -n kuadrant-system -l app=authorino --tail=50
```

- Verify that the **MaaSModelRef** exists for the model:

```
$ oc get maasmodelref -n <model-namespace>
```

1.15.6. Subscription management issues

If users receive access errors when attempting to use models after creating a subscription:

- Verify that the user's OpenShift groups are listed in the subscription's groups.
- Verify that the model is included in the subscription's model list.
- Check that at least one token limit is configured for each model in the subscription.
- If multiple subscriptions apply to a user, verify that the correct subscription is being used based on priority level (higher numbers have higher priority).
- Check the MaaS API logs for subscription resolution errors:

```
$ oc logs -n redhat-ods-applications -l app.kubernetes.io/name=maas-api --tail=50 | grep subscription
```

- Verify that a matching authorization policy was created if you selected that option during subscription creation.

1.15.7. Subscription phase shows Failed

If a subscription shows a **Failed** phase status:

- Check the subscription status conditions for the failure reason:

```
$ oc describe maassubscription <subscription-name> -n redhat-ods-applications
```

- Verify that all referenced models exist and have valid **MaaSModelRef** objects.
- Ensure that the groups specified in the subscription are valid OpenShift groups.
- Check that token limits are properly configured for all models.
- Review the MaaS API logs for detailed error messages:

```
$ oc logs -n redhat-ods-applications -l app.kubernetes.io/name=maas-api --tail=100
```

CHAPTER 2. USE MODELS-AS-A-SERVICE

Use Models-as-a-Service (MaaS) to access large language models with subscription-based governance and self-service API key management. You can discover available models, create API keys, and integrate models into your applications using OpenAI-compatible APIs.

2.1. FIND MODELS-AS-A-SERVICE MODELS IN THE DASHBOARD

In Red Hat OpenShift AI, models published to Models-as-a-Service (MaaS) are available through the **AI asset endpoints** page in the OpenShift AI dashboard.

To view available models, log in to the OpenShift AI dashboard and click **Gen AI studio → AI asset endpoints**.

The **Models** tab displays a table with all available model deployments. Each row shows:

Model

The model deployment name and model ID.

Use case

The model type, such as LLM for large language models.

Status

The current operational state of the model: **Ready**, **Not ready**, or **Unknown**.

Endpoints

A **View** button that opens the endpoint details.

Playground

An **Add to playground** link for testing the model interactively. For more information, see Test models in the playground.

To identify whether a model is published to Models-as-a-Service (MaaS), click the **View** button under Endpoints. Models published to MaaS display a **Model as a Service** badge at the top of the Endpoints dialog. The dialog shows:

- The API endpoint URL for accessing the model through the MaaS gateway from outside the cluster
- A subscription selector for choosing which subscription to use
- A **Generate API key** button for creating 1-hour temporary API keys
- A link to the **API Keys** page for managing API keys

To manage your API keys for programmatic access to MaaS models, click **Gen AI studio → API keys**.

Next steps

- Access models through models-as-a-service

Additional resources

- Models-as-a-Service overview
- Create an API key

- Test models in the playground
- View subscription limits

2.2. ACCESS MODELS THROUGH MODELS-AS-A-SERVICE

In Red Hat OpenShift AI, you can use Models-as-a-Service (MaaS) to access large language models with subscription-based governance and self-service API key management.

2.2.1. Your token limits and access levels

Before you begin working with models, it's helpful to understand how Models-as-a-Service (MaaS) uses subscriptions and authorization policies to control your quota and access.

NOTE

In OpenShift AI 3.4, MaaS uses subscriptions instead of tiers and API keys instead of service account tokens for authentication.

- **Subscription assignment**: You are automatically assigned to subscriptions based on your group membership in OpenShift. If you belong to multiple groups with different subscriptions, you can access all those subscriptions. When creating an API key without specifying a subscription, the system selects the subscription with the highest priority level.
- **Token limits**: Your subscription determines how many tokens you can consume per time period for each model.
- **Model access**: Authorization policies determine which models you can access through the API gateway. Your administrator creates authorization policies that grant your groups access to specific model endpoints. Both a subscription with quota and an authorization policy are required to access models through MaaS.
- **Authentication**: You must use an API key to access models. You can create and manage your own API keys through the dashboard.

2.2.2. View your subscriptions

In Red Hat OpenShift AI, you can see which Models-as-a-Service (MaaS) subscriptions you can use for a specific model from the **AI asset endpoints** page.

Prerequisites

- You have access to the OpenShift AI dashboard.
- At least one model has been published to Models-as-a-Service in your environment.
- You belong to a group that is assigned to at least one subscription.

Procedure

1. In the OpenShift AI dashboard, click **Gen AI studio → AI asset endpoints**.
2. On the **Models** tab, locate a model published to Models-as-a-Service (MaaS) and click **View** in the **Endpoints** column.

3. In the **Endpoints** dialog, view your subscription assignment from the **Subscription** dropdown. If you belong to multiple subscriptions that include this model, you can select which subscription to use for the model. The selected subscription's token limits apply to your inference requests.

   TIP

   If you frequently exceed token limits or need access to additional models, contact your administrator to request a subscription update.

Verification

- The **Subscription** dropdown lists at least one subscription for which you have access to this model.

2.2.3. Generate a temporary API key

In Red Hat OpenShift AI, you can generate a temporary 1-hour API key directly from the **Endpoints** dialog for quick testing and prototyping of Models-as-a-Service (MaaS) endpoints.

Prerequisites

- You have access to the OpenShift AI dashboard.
- Your administrator has enabled Models-as-a-Service and assigned you to a subscription.

Procedure

1. In the OpenShift AI dashboard, click **Gen AI studio → AI asset endpoints**.
2. On the **Models** tab, locate the model you want to access and click **View** in the **Endpoints** column.
3. In the **Endpoints** dialog, verify that the model displays a **Model as a Service** badge at the top.
4. Copy the external API endpoint URL and store it securely. This URL is required for API calls to the model.
5. Under the **Authentication** section, select your subscription from the **Subscription** dropdown.
6. Click **Generate API key**.
7. Copy the generated API key immediately and store it securely.

   IMPORTANT

   The temporary API key is displayed only once and expires after 1 hour. Temporary API keys generated from the **Endpoints** dialog are scoped to the selected subscription. To create API keys with custom expiration dates, navigate to **Gen AI studio → API keys** in the OpenShift AI dashboard.

8. Click **Close**.

Verification

- The generated API key displays with the **sk-oai-** prefix.
- You have securely stored both the API key and the external API endpoint URL for use in API calls.

Next steps

- Make API calls to models

2.2.4. Make API calls to models

In Red Hat OpenShift AI, you can use your Models-as-a-Service (MaaS) API key to list models available to you and to make inference requests through the OpenAI-compatible API.

Prerequisites

- You have generated a temporary API key or created an API key. For more information, see Create an API key.
- You have copied the external API endpoint URL.
- At least one model is published to Models-as-a-Service and accessible through your subscription.

Procedure

1. Set your API key as an environment variable:

```
$ export MAAS_API_KEY="<your_api_key>"
```

The command uses the following placeholder:

<your_api_key>
Specifies the API key you generated or created, with the **sk-oai-** prefix.

2. Set the external API endpoint URL as an environment variable:

```
$ export MAAS_URL="<external_api_endpoint>"
```

The command uses the following placeholder:

<external_api_endpoint>
Specifies the external API endpoint URL you copied from the **Endpoints** dialog.

3. List available models using the **/v1/models** endpoint:

```
$ curl -X GET "${MAAS_URL}/v1/models" \
  -H "Authorization: Bearer ${MAAS_API_KEY}"
```

Example response in OpenAI-compatible format:

```
{
  "object": "list",
  "data": [
```

```
    {
      "id": "facebook-opt-125m",
      "object": "model",
      "created": 1234567890,
      "owned_by": "llm",
      "ready": true
    },
    {
      "id": "llama-2-7b-chat",
      "object": "model",
      "created": 1234567890,
      "owned_by": "llm",
      "ready": true
    }
  ]
}
```

4. Call a model using the chat completions endpoint:

```
$ curl -X POST "${MAAS_URL}/llm/<model_name>/v1/chat/completions" \
  -H "Authorization: Bearer ${MAAS_API_KEY}" \
  -H "Content-Type: application/json" \
  -d { "model": "<model_name>", "messages": [ { "role": "user", "content": "Explain quantum computing in simple terms." } ], "max_tokens": 150, "temperature": 0.7 }
```

where:

<model_name>

Specifies the model deployment name from the Models as a service page, such as **facebook-opt-125m** or **llama-2-7b-chat**.

Example response:

```
{
  "id": "chatcmpl-abc123",
  "object": "chat.completion",
  "created": 1234567890,
  "model": "llama-2-7b-chat",
  "choices": [
    {
      "index": 0,
      "message": {
        "role": "assistant",
        "content": "Quantum computing is a type of computing that uses quantum mechanics..."
      },
      "finish_reason": "stop"
    }
  ],
  "usage": {
    "prompt_tokens": 12,
    "completion_tokens": 45,
    "total_tokens": 57
  }
}
```

Verification

- The /**v1**/**models** response includes the models you expect to access.
- Chat completion requests return responses in the OpenAI-compatible JSON format.

Troubleshooting

- If you receive **401 Unauthorized**, verify that your API key is valid and has not been revoked.
- If you receive **429 Too Many Requests**, your subscription's token limit has been reached. Wait for the token limit window to reset.

2.2.5. Test models in a Jupyter notebook

In Red Hat OpenShift AI, you can test Models-as-a-Service endpoints in a Jupyter notebook using Python code. This approach is useful for iterative testing, prototyping model integrations, and validating rate limiting behavior in an interactive environment before integrating models into applications.

Prerequisites

- You have generated a temporary API key or created a permanent API key for the model that you want to access.
- You have copied the external API endpoint URL from the **Endpoints** dialog. The procedure describes how to extract the base gateway URL.
- You have access to a Jupyter notebook environment in OpenShift AI.
- You have Python 3.9 or later available in your notebook environment.

Procedure

1. Create a new Jupyter notebook in your OpenShift AI workbench.
2. In the first code cell, configure the Models-as-a-Service (MaaS) gateway URL and API key:

```
DEMO_MAAS_BASE = "https://maas.example.com"
DEMO_API_KEY = "your-api-key-here"
```

   The code uses the following variables:

   DEMO_MAAS_BASE

   Specifies the MaaS gateway base URL. Extract this from the external API endpoint URL you copied from the **Endpoints** dialog. For example, if the endpoint is **https://maas.apps.cluster.example.com/llm/facebook-opt-125m**, use **https://maas.apps.cluster.example.com**. The specific model URLs are retrieved from the API response in the next step.

   DEMO_API_KEY

   Specifies your API key. Use the temporary or permanent key you generated earlier.

3. In a new code cell, add the setup code to configure the HTTP client:

```
import json
```

```
import os
import ssl
import urllib.error
import urllib.request
from typing import Any, Dict, Optional

_mb = globals().get("DEMO_MAAS_BASE", "")
if isinstance(_mb, str) and _mb.strip():
    MAAS_BASE = _mb.strip().rstrip("/")
else:
    MAAS_BASE = os.environ.get("MAAS_BASE",
"https://maas.YOUR_DOMAIN_HERE").strip().rstrip("/")

_ak = globals().get("DEMO_API_KEY", "")
if isinstance(_ak, str) and _ak.strip():
    API_KEY = _ak.strip()
else:
    API_KEY = (os.environ.get("MAAS_API_KEY") or os.environ.get("API_KEY") or "").strip()

VERIFY_TLS = os.environ.get("VERIFY_TLS", "").lower() in ("1", "true", "yes")
MODELS_URL = f"{MAAS_BASE}/maas-api/v1/models"

if not API_KEY:
    raise SystemExit(
        "Set DEMO_API_KEY in the quick-swap cell or MAAS_API_KEY / API_KEY in the
environment."
    )


def http_json(
    method: str,
    url: str,
    *,
    token: Optional[str] = None,
    data: Optional[Dict[str, Any]] = None,
):
    """Minimal JSON HTTP helper (stdlib only)."""
    headers = {"Content-Type": "application/json", "Accept": "application/json"}
    if token:
        headers["Authorization"] = f"Bearer {token}"
    body = None
    if data is not None:
        body = json.dumps(data).encode("utf-8")
    ctx = ssl.create_default_context()
    if not VERIFY_TLS:
        ctx.check_hostname = False
        ctx.verify_mode = ssl.CERT_NONE
    req = urllib.request.Request(url, data=body, headers=headers, method=method)
    try:
        with urllib.request.urlopen(req, context=ctx, timeout=120) as resp:
            raw = resp.read().decode("utf-8")
            return resp.status, json.loads(raw) if raw else {}
    except urllib.error.HTTPError as e:
        err_body = e.read().decode("utf-8", errors="replace")
        try:
            parsed = json.loads(err_body) if err_body else {}
```

```
        except json.JSONDecodeError:
            parsed = {"_raw": err_body}
        raise RuntimeError(f"HTTP {e.code}: {parsed}") from None


print("MAAS_BASE :", MAAS_BASE)
print("MODELS_URL:", MODELS_URL)
print("VERIFY_TLS:", VERIFY_TLS)
print("API key set:", bool(API_KEY))
```

This cell defines the **http_json** helper function and prints the configuration summary.

4. In a new code cell, list the available models:

```
_, models_body = http_json("GET", MODELS_URL, token=API_KEY)

data = models_body.get("data") or []
if not data:
    raise SystemExit("No models in response; deploy a model and check subscription
binding.")

first = data[0]
MODEL_NAME = first.get("id") or first.get("name")
MODEL_URL = (first.get("url") or "").rstrip("/")
if not MODEL_NAME or not MODEL_URL:
    raise RuntimeError(f"Could not parse model id/url from: {first}")

print(f"MODEL_NAME: {MODEL_NAME}")
print(f"MODEL_URL: {MODEL_URL}")
print("\nAll models:")
print(json.dumps(models_body, indent=2))
```

This cell retrieves the list of models you can access and stores the first model's name and URL for testing.

5. In a new code cell, make a single inference request:

```
COMPLETIONS_URL = f"{MODEL_URL}/v1/completions"
inference_payload = {
    "model": MODEL_NAME,
    "prompt": "Hello from the notebook demo.",
    "max_tokens": 50,
}

status, completion = http_json("POST", COMPLETIONS_URL, token=API_KEY,
data=inference_payload)

usage = completion.get("usage") or {}
choices = completion.get("choices") or []
choice0 = choices[0] if choices and isinstance(choices[0], dict) else {}
completion_text = choice0.get("text") or ""

print(f"HTTP status: {status}")
print(f"\nTokens used:")
print(f"  prompt_tokens: {usage.get('prompt_tokens', '—')}")
print(f"  completion_tokens: {usage.get('completion_tokens', '—')}")
```

```
print(f"  total_tokens: {usage.get('total_tokens', '—')}")
print(f"\nCompletion text:")
print(completion_text if completion_text else "(empty)")
```

This cell sends a completion request and displays the response with token usage information.

6. Optional: In a new code cell, test the rate limiting behavior:

```
import time

# Configuration
SLEEP_BETWEEN_REQUESTS_SEC = 1.0
CONSECUTIVE_429_TO_STOP = 3
MAX_REQUESTS = 64

inference_payload = {
    "model": MODEL_NAME,
    "prompt": "Rate limit probe.",
    "max_tokens": 50,
}

ctx = ssl.create_default_context()
if not VERIFY_TLS:
    ctx.check_hostname = False
    ctx.verify_mode = ssl.CERT_NONE

consecutive_429 = 0
last_code = None
for i in range(1, MAX_REQUESTS + 1):
    body = json.dumps(inference_payload).encode("utf-8")
    req = urllib.request.Request(
        COMPLETIONS_URL,
        data=body,
        headers={
            "Authorization": f"Bearer {API_KEY}",
            "Content-Type": "application/json",
        },
        method="POST",
    )
    try:
        with urllib.request.urlopen(req, context=ctx, timeout=120) as resp:
            last_code = resp.status
    except urllib.error.HTTPError as e:
        last_code = e.code

    print(f"{i:3d}  HTTP {last_code}", end="")
    if last_code == 429:
        consecutive_429 += 1
        print(f"  (429 streak {consecutive_429}/{CONSECUTIVE_429_TO_STOP})")
        if consecutive_429 >= CONSECUTIVE_429_TO_STOP:
            print(f"Stopping: {CONSECUTIVE_429_TO_STOP} consecutive 429 responses.")
            break
    else:
        consecutive_429 = 0
        print()
```

```
    if SLEEP_BETWEEN_REQUESTS_SEC > 0 and i < MAX_REQUESTS:
        time.sleep(SLEEP_BETWEEN_REQUESTS_SEC)
else:
    if consecutive_429 < CONSECUTIVE_429_TO_STOP:
        print(f"Stopped: reached MAX_REQUESTS={MAX_REQUESTS} without
{CONSECUTIVE_429_TO_STOP} consecutive 429s.")
```

This cell sends repeated requests to test rate limiting. The loop stops after receiving 3 consecutive HTTP 429 responses or reaching the maximum request count.

Verification

- The setup cell prints your MaaS configuration without exposing the API key value.
- The model list cell displays at least one available model with its name and URL.
- The inference cell returns an HTTP 200 status with completion text and token usage statistics.
- The rate limit test demonstrates throttling behavior by receiving HTTP 429 responses when limits are exceeded.

Additional resources

- Models-as-a-Service token limit responses

2.2.6. Token limit responses

In Red Hat OpenShift AI, when you exceed token limits for your Models-as-a-Service (MaaS) subscription, the API returns specific error responses to help you manage your usage.

Token limit exceeded

If you exceed the maximum number of tokens allowed per time period for a model in your subscription:

Example error response

```
{
  "error": {
    "message": "Token limit exceeded. You have consumed the maximum number of tokens allowed
for this model in your subscription.",
    "type": "token_limit_error",
    "code": 429
  }
}
```

Response headers for retry logic

```
X-RateLimit-Limit: 1000
X-RateLimit-Remaining: 0
X-RateLimit-Reset: 1234567890
Retry-After: 42
```

Handling token limits in Python

```
import time
```

```
import requests

def make_request_with_retry(url, headers, data, max_retries=3):
    for attempt in range(max_retries):
        response = requests.post(url, headers=headers, json=data)

        if response.status_code == 429:
            retry_after = int(response.headers.get('Retry-After', 60))
            print(f"Token limit exceeded. Retrying after {retry_after} seconds...")
            time.sleep(retry_after)
            continue

        return response

    raise Exception("Max retries exceeded")
```

Mitigation strategies

- Reduce **max_tokens** in your requests to consume fewer tokens per request
- Implement exponential backoff retry logic to handle temporary limit violations
- Batch requests with longer delays between them to spread token consumption over time
- Contact your administrator to request a subscription update with higher token limits if you consistently hit limits

2.2.7. Test models in the playground

In Red Hat OpenShift AI, the **Gen AI Studio playground** provides a chat-style interface for sending prompts to deployed models and reviewing the responses. You can test Models-as-a-Service (MaaS) endpoints in the playground using your existing subscriptions and token quotas.

IMPORTANT

Playground testing consumes tokens from your subscription's token limit. Heavy testing can deplete your quota and block subsequent production usage. Use a subscription dedicated to development if available.

Prerequisites

- You are logged in to the OpenShift AI dashboard.
- You are a member of at least one group that is granted access to a Models-as-a-Service (MaaS) subscription.
- At least one model has been published to MaaS and is included in your subscription.
- You have deployed a Llama Stack server in your project. For more information, see Deploying a Llama Stack server.

Procedure

1. From the OpenShift AI dashboard, click **Gen AI studio → AI asset endpoints**.

2. Locate the MaaS model that you want to test.

3. In the **Actions** column for that model, click the action menu (⋮) and then select **Try in playground**.
   The playground interface opens in a new browser tab.

4. In the **Configure** panel, verify your MaaS settings:

   a. From the **Model** dropdown, verify that the correct model is selected.
      MaaS models are displayed in the format **<endpoint-name>/<model-id>**. For example, **maas-vllm-inference-1/facebook/opt-125m**.

   b. From the **Subscription** dropdown, select the subscription to use for testing.
      If you belong to multiple subscriptions, select the subscription you want to use for testing.

5. Test the model by entering a prompt in the message field and then clicking **Send**.
   The model response appears in the chat interface.

6. Optional: Adjust model parameters, configure system prompts, or use additional playground features.
   For information about playground features such as temperature settings, streaming responses, system prompts, model comparison, RAG integration, and code export, see Experimenting with models in the gen AI playground.

Verification

- The model returns a response to your prompt.

Additional resources

- Experimenting with models in the gen AI playground
- View subscription limits

2.2.8. Manage persistent API keys

2.2.8.1. View your API keys

In Red Hat OpenShift AI, you can view a list of your Models-as-a-Service (MaaS) API keys in the OpenShift AI dashboard. The list shows the status, creation date, last-used date, and expiration date for each key.

Prerequisites

- You are logged in to the OpenShift AI dashboard.
- Your administrator has enabled Models-as-a-Service and assigned you to a subscription.

Procedure

1. In the OpenShift AI dashboard, click **Gen AI studio → API keys**.
   The **API keys** page displays a table with the following columns:

   - **Name**: The name assigned to the API key

- **Status**: The current state of the key. Possible values: **Active**, **Expired**, **Revoked**.
- **Owner**: Your username
- **Creation date**: The date when the key was created
- **Last used**: The date when the key was last used to access a model
- **Expiration date**: The expiration date for the key

2. Optional: Filter the list of API keys by status using the **Status** dropdown and selecting **Active**, **Expired**, or **Revoked**.
3. Optional: Sort the table by clicking any column header.

Verification

- Verify that the **API keys** table displays your API keys with columns for name, status, owner, creation date, last-used date, and expiration date.
- If you applied status filters, verify that the table shows only API keys matching the selected status.

2.2.8.2. Create an API key

In Red Hat OpenShift AI, you can create Models-as-a-Service (MaaS) API keys to authenticate inference requests to large language models.

IMPORTANT

Your group membership is captured at API key creation time. If your group membership changes after the key is created, the key retains the original group associations. To reflect updated group membership, revoke the existing key and create a new one.

Prerequisites

- You are logged in to the OpenShift AI dashboard.
- You authenticate to OpenShift AI using OpenShift authentication. External OIDC users create API keys through the MaaS API using **curl** or other HTTP clients, not through the dashboard.
- Your administrator has deployed Models-as-a-Service.
- You are a member of at least one OpenShift group that is included in a MaaS subscription.

Procedure

1. In the OpenShift AI dashboard, click **Gen AI studio → API keys**.
2. Click **Create API key**.
3. In the **Create API key** dialog, configure the following settings:
   a. In the **Name** field, enter a descriptive name for the API key.
   b. Optional: In the **Description** field, enter additional details about the key's purpose.

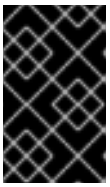
c. From the **Subscription** dropdown, select the subscription that determines which models the key can access and the applicable token limits.
The **Models** section displays the models included in the selected subscription and their configured token limits.

d. From the **Expiration** dropdown, select the number of days until the key expires, or select **Never** to create a permanent key.

NOTE

You can select an expiration period from 1 to 365 days. The default expiration is 30 days. Your administrator can set a maximum expiration limit in the **Tenant** custom resource. If not set, the default maximum is 90 days.

4. Click **Create**.
The **API key created** dialog displays the generated key with a prefix of **sk-oai-**.



IMPORTANT

The plaintext key is displayed only during creation and cannot be retrieved later. Save the key in a secrets manager before closing the **API key created** dialog. If you lose the key, you must revoke it and create a new one.

5. Click the copy icon next to the API key field to copy the key, and then save it in a secure location for use in applications.

6. Click **Close**.

Verification

1. In the OpenShift AI dashboard, navigate to **Gen AI studio → API keys**.

2. Verify that the new API key appears in the table.

3. Confirm that the **Status** column displays **Active** with a green checkmark.

4. Verify that the **Subscription** column shows a subscription that includes the models you want to access.

5. Check that the **Expires** column displays the correct expiration date based on the number of days you selected.

6. Optional: Test the API key by listing the models available through your subscription:

```
$ curl -H "Authorization: Bearer <your-api-key>" \
  https://<maas-gateway-url>/maas-api/v1/models
```

The command uses the following placeholders:

<your-api-key>

Specifies the API key you created.

<maas-gateway-url>

Specifies your MaaS gateway URL.

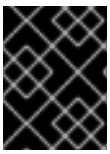
The response lists the models accessible through your subscription in JSON format.

Next steps

- Make API calls to models

2.2.8.3. Revoke your API key

In Red Hat OpenShift AI, you can revoke one of your Models-as-a-Service (MaaS) API keys if you no longer need it or suspect that it has been compromised.



IMPORTANT

Revoking an API key is permanent and cannot be undone. Applications and services using the revoked key lose access and receive **401 Unauthorized** responses.

Prerequisites

- You have access to the OpenShift AI dashboard.
- You have at least one API key.
- The key you want to revoke has not already been revoked or expired.

Procedure

To revoke an individual API key:

1. In the OpenShift AI dashboard, click **Gen AI studio → API keys**.
2. In the row for the API key you want to revoke, click the action menu (⋮) and select **Revoke**.
3. In the **Revoke API key?** dialog, review the warning that revocation is permanent.
4. Enter the API key name to confirm.
5. Click **Revoke**.

To revoke all your API keys:

1. In the OpenShift AI dashboard, click **Gen AI studio → API keys**.
2. Click the action menu (⋮) and select **Revoke all my keys**.
3. In the dialog, review the warning that revocation is permanent.
4. Click **Revoke all keys**.

Verification

- The API key shows a **Revoked** status in the API keys table. The revoked key remains visible in the table but can no longer be used for authentication.
- Attempting to use the revoked key in an API request returns a **401 Unauthorized** response with the error code **invalid_api_key**.

Next steps

- Create an API key
- View your API keys

2.2.9. Best practices for using Models-as-a-Service

As a OpenShift AI user, follow these recommendations to effectively and securely use Models-as-a-Service (MaaS).

2.2.9.1. API key security

- Never commit API keys to version control. Store API keys in environment variables, Kubernetes secrets, or secret management systems such as HashiCorp Vault.
- Use descriptive names for API keys. Create separate keys for different applications or purposes such as **production-chatbot**, **dev-testing**, or **notebook-experiments** to track usage and simplify revocation.
- Rotate API keys periodically. Generate new keys regularly and revoke old ones, especially for long-lived keys used in production applications.
- Revoke compromised keys immediately. If an API key is exposed or no longer needed, revoke it through the dashboard to prevent unauthorized access.
- Set appropriate expiration dates. Use shorter expiration periods for development and testing, and coordinate with your team for production key rotation schedules.

2.2.9.2. Quota management

- Monitor your subscription limits. Check your subscription quotas in the dashboard to understand your available token limits and avoid unexpected quota exhaustion.
- Check rate limit headers in responses. Review the **X-RateLimit-Remaining** header in API responses to track your usage against subscription limits in real time.
- Plan for quota exhaustion. Implement graceful error handling when quotas are depleted, such as queuing requests or displaying user-friendly messages.
- Choose subscriptions wisely. If you belong to multiple subscriptions with access to the same model, select the appropriate subscription for your use case when creating API keys or testing in the playground.

2.2.9.3. Token optimization

- Set reasonable **max_tokens** values. Request only as many tokens as you need for your use case. Avoid setting excessively high limits that waste quota.
- Optimize prompts for efficiency. Shorter, more focused prompts often produce better results while consuming fewer tokens than verbose or repetitive prompts.
- Use system prompts effectively. Configure consistent model behavior through system prompts rather than repeating instructions in every user message.

- Avoid redundant context. Send only the conversation history that the model needs to respond to the current request.
- Test prompts before deploying. Use the playground to refine prompts and verify token consumption before implementing them in production code.

2.2.9.4. Performance and reliability

- Implement retry logic with exponential backoff. Handle rate limit errors (HTTP 429) and temporary failures gracefully by retrying requests with increasing delays.
- Cache responses when appropriate. If you make identical requests repeatedly, cache the results to reduce token consumption and improve response time.
- Use streaming for interactive applications. Enable streaming responses to provide faster time-to-first-token and better user experience for chat-based applications.
- Handle errors gracefully. Implement proper error handling for rate limits, quota exhaustion, network errors, and model timeouts.

2.2.9.5. Testing and development

- Test in the playground first. Use the playground to validate model responses, compare models, and experiment with parameters before writing code.
- Export playground configurations. Use the playground's code export feature to generate starter code templates with your tested prompts and parameters.
- Use temporary API keys for development. Generate short-lived API keys for testing and experimentation to minimize security risk if keys are accidentally exposed.
- Validate model responses. Implement response validation in your application to ensure the model's output meets your requirements and handles edge cases.
- Test with multiple models. If multiple models are available in your subscription, test different models to find the best balance of response quality, speed, and token consumption for your use case.

2.2.9.6. Multi-subscription usage

- Understand subscription priorities. If you belong to multiple subscriptions with access to the same model, the subscription with the highest priority level is used by default for API requests.
- Select subscriptions explicitly when needed. When creating API keys or testing in the playground, explicitly choose which subscription to use rather than relying on automatic priority-based selection.
- Separate development and production usage. If possible, use different subscriptions for development, testing, and production to isolate quota consumption and costs.
- Track your usage per subscription. Monitor your token consumption separately for each subscription you belong to, especially if subscriptions have different usage policies or billing.

2.2.9.7. Production deployment

- Use descriptive API key names. Create separate API keys for each production application or service to support usage tracking and selective revocation.
- Implement monitoring and logging. Log API requests and responses (excluding sensitive data) to support debugging, usage analysis, and compliance requirements.
- Set up alerting for quota limits. Monitor your subscription quota consumption and alert your team when approaching limits to avoid service disruptions.
- Coordinate with administrators. If your usage patterns change or you need higher quotas, contact your administrator to request subscription adjustments.

Additional resources

- View subscription limits
- Create API keys
- Make API calls to Models-as-a-Service (MaaS) models
- Experimenting with models in the gen AI playground

Additional resources

- Models-as-a-Service user access troubleshooting

2.3. MODELS-AS-A-SERVICE USER ACCESS TROUBLESHOOTING

As a OpenShift AI user, you can diagnose and resolve common issues when accessing models through Models-as-a-Service (MaaS).

2.3.1. Authentication errors: 401 Unauthorized

Symptom

```
{
  "error": {
    "message": "Invalid or expired API key",
    "type": "authentication_error",
    "code": 401
  }
}
```

Possible causes and solutions

- **API key expired**: If your API key has expired, create a new one from the API keys page.
- **API key revoked**: Check if the API key has been revoked. Create a new API key if necessary.
- **Incorrect API key format** Ensure you're using the **Authorization: Bearer <key>** header format with the full API key including the **sk-oai-** prefix.
- **Using wrong authentication method** Generate a MaaS API key from the dashboard. OpenShift tokens are not valid for MaaS authentication.

2.3.2. Authorization errors: 403 Forbidden

Symptom

```
{
  "error": {
    "message": "Access denied. Your subscription does not have permission to access this model.",
    "type": "authorization_error",
    "code": 403
  }
}
```

Possible causes and solutions

- **Model not included in your subscription** Contact your administrator to request that the model be added to your subscription.
- **No authorization policy exists**: Ask your administrator to verify that an authorization policy exists for your groups to access the model through the API gateway.

2.3.3. Model not found: 404

Symptom

```
{
  "error": {
    "message": "Model not found",
    "type": "not_found_error",
    "code": 404
  }
}
```

Possible causes and solutions

- **Incorrect model name**: Verify the model name using the **/v1/models** endpoint.
- **Model not deployed**: Ask your administrator to check if the model is deployed and ready.
- **Typo in URL**: Ensure the URL format is correct: **https://maas.<domain>/llm/<model-name>/v1/chat/completions**

2.3.4. Exceeded token limits

If you exceed the maximum number of tokens allowed per time period for a model in your subscription:

Example error response

```
{
  "error": {
    "message": "Token limit exceeded. You have consumed the maximum number of tokens allowed for this model in your subscription.",
    "type": "token_limit_error",
```

```
    "code": 429
  }
}
```

Response headers for retry logic

```
X-RateLimit-Limit: 1000
X-RateLimit-Remaining: 0
X-RateLimit-Reset: 1234567890
Retry-After: 42
```

Handling token limits in Python

```
import time
import requests

def make_request_with_retry(url, headers, data, max_retries=3):
    for attempt in range(max_retries):
        response = requests.post(url, headers=headers, json=data)

        if response.status_code == 429:
            retry_after = int(response.headers.get('Retry-After', 60))
            print(f"Token limit exceeded. Retrying after {retry_after} seconds...")
            time.sleep(retry_after)
            continue

        return response

    raise Exception("Max retries exceeded")
```

Mitigation strategies

- Reduce **max_tokens** in requests to consume fewer tokens per request
- Implement exponential backoff retry logic to handle temporary limit violations
- Batch requests with longer delays between them to spread token consumption over time
- Contact your administrator to request a subscription update with higher token limits if you consistently hit limits

2.3.5. Persistent token limit errors

Symptom

You continue to receive 429 errors even after waiting.

Possible causes and solutions

- **Subscription token limits exhausted**: Check token limits for your subscription for the model you are trying to access. Wait for the time period to reset or contact your administrator for a subscription update.
- **Incorrect retry logic**: Ensure you're respecting the **Retry-After** header value.

- **Multiple applications using the same API key** Each application should have its own API key to better track and manage token consumption.